



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Dipartimento di Ingegneria Elettrica e Tecnologie Dell'Informazione

BugBoard2026

Progetto di Ingegneria del Software
Anno Accademico 2025/2026

Noemi Spera

Matricola: *N86003717*

Michele Pio Loreto

Matricola: *N86003803*

Napoli, maggio 2026

Indice

Introduzione	i
BugBoard26	i
Struttura della documentazione	i
 I Documento di Analisi e Specifica dei Requisiti	 1
1 Requisiti funzionali	2
1.1 Definizione dei casi d'uso	2
1.2 Descrizione dettagliata dei casi d'uso	2
1.3 Modellazione dei casi d'uso	4
1.4 Formalizzazione casi d'uso	5
1.4.1 Segnalare una issue	5
1.4.2 Modifica di un progetto	7
1.4.3 Aggiungere un commento ad una issue	9
1.5 Prototipazione Visuale: MockUp	11
1.5.1 Modifica di un progetto	11
1.5.2 Segnalare una issue	12
1.5.3 Aggiungere un commento ad una issue	13
 2 Requisiti non funzionali	 15
2.1 Usabilità	15
2.2 Sicurezza	15
2.3 Portabilità	16
 3 Requisiti di dominio	 17
 4 Target Utenti	 18
4.1 Personas	18
4.1.1 Tratti caratteristici	20
 II Documento di Design	 21
 5 Architettura del sistema	 22
5.1 Analisi dell'architettura	22

5.1.1	Client	22
5.1.2	Server	23
6	Tecnologie e strumenti di sviluppo	25
6.1	Architettura Backend e Persistenza dei Dati	25
6.1.1	Node.js & Express	25
6.1.2	PostgreSQL & Sequelize	25
6.2	Sviluppo Frontend e Interfaccia Utente	26
6.2.1	Angular	26
6.2.2	Tailwind CSS	26
6.2.3	Toastr Notifications	26
6.3	Qualità del Codice e Testing Strutturato	26
6.3.1	SonarQube	26
6.3.2	Jest Testing	26
6.3.3	Playwright	27
6.4	Infrastruttura, DevOps e Ciclo di Vita del Software	27
6.4.1	Docker	27
6.4.2	Azure	29
7	Persistenza dati	30
7.1	Descrizione dello schema per la persistenza dati	30
8	Diagrammi UML delle classi di design	32
8.1	Diagrammi delle classi di design	32
8.1.1	Autenticazione e gestione delle utenze	32
8.1.2	Autenticazione	33
8.2	Gestione dei progetti	34
8.3	Gestione delle issue e commenti	35
8.4	Sequence Diagram	36
8.4.1	Creazione di una issue	36
8.5	Statechart	37
8.5.1	Gestione dello stato di una issue	37
9	Scelte di design dell'interfaccia utente	38
9.1	Font e colori	38
9.2	Principi della Gestalt	39
9.2.1	Principio di Prossimità	39
9.2.2	Euristiche di Nielsen	40
9.2.3	Feedback continuo	40
9.2.4	Consistenza	40
10	Strumenti di versioning	41
11	SonarQube - Analisi della qualità del codice	42
11.0.1	Report Finale	42
11.0.2	Grafico delle issues	43

12 Azure - Deploy dell'applicazione	44
12.0.1 Architettura scelta	44
 III Attività di Testing	 46
13 Test del sistema	47
13.1 Test di Unità	47
13.1.1 Testing Black Box del metodo creaUser()	47
13.1.2 Testing black-box del metodo findIssueByTag()	55
13.1.3 Testing white-box del metodo getAllProjects()	60
13.2 Test End-to-End	67
13.2.1 Setup dell'ambiente di test	67
13.2.2 Test di Autenticazione Utente	67
13.2.3 Test di Creazione di una Nuova Issue	69
13.2.4 Test di Creazione di un Nuovo Progetto	71
13.2.5 Test di Creazione di un Nuovo Commento	73
13.2.6 Test di Ricerca delle Issue tramite Tag	75
 14 Test di usabilità	 78
14.0.1 Esito della analisi tramite Expert Review	79
 15 Esperimenti con utenti reali	 80
15.1 Candidati	80
15.1.1 Definizione dei task	80
15.1.2 Risultati dell'esperimento	82
15.2 Sondaggio finale	83
15.3 Conclusioni	84

Elenco delle figure

1.1	Diagramma dei casi d'uso	4
1.2	MockUp_01 Project Update	11
1.3	MockUp_02 NewIssue	12
1.4	MockUp_03 AddNewComment	13
4.1	Personas_01 Project Manger	18
4.2	Personas_02 Developer	19
4.3	Personas_03 Tester	19
8.1	Diagramma delle classi del Login	33
8.2	Diagramma delle classi per la gestione dei progetti	34
8.3	Diagramma delle classi per la gestione delle issue	35
8.4	Diagramma di sequenza per la creazione di una issue	36
8.5	Diagramma degli stati per l'aggiornamento dello stato di una issue	37
9.1	Interfaccia di login	38
9.2	Visualizzazione bottoni	39
9.3	Interfaccia lista issue	39
9.4	Interfaccia lista progetti	39
9.5	Disposizione bottoni nelle interfacce di crezione	40
10.1	Interfaccia di github	41
11.1	SonarQube-Report finale	42
11.2	SonarQube-Andamento temporale delle issue	43
15.1	Sondaggio test usabilità	83

Elenco delle tabelle

1.1	Tabella dei casi d'uso	2
12.1	Tabella mapping tra componente app e Servizio cloud usato	44
13.1	Specifica dei casi di test logici per la creazione utente	48
13.2	Specifica dei casi di test logici per la ricerca di una issue	55
13.3	Percorsi Indipendenti	61
13.4	Specifica dei casi di test logici per i progetti di un utente	62
15.1	Tabella dei candidati per testare l'usabilità	81
15.2	Metriche raccolte durante i test di usabilità	82

Introduzione

BugBoard26

BugBoard26 è una piattaforma per la **gestione collaborativa** di issue nei progetti software. Il sistema consente ai team di sviluppo di segnalare anomalie relative a un progetto e di monitorarne costantemente lo stato di avanzamento. La piattaforma promuove la partecipazione attiva del team: gli utenti possono collaborare alla risoluzione fornendo **suggerimenti** e **dettagli tecnici** tramite i commenti. Tuttavia, per garantire il controllo sul ciclo di vita della segnalazione, solo l'utente creatore possiede i permessi necessari per chiudere e contrassegnare la issue come definitivamente risolta.

Struttura della documentazione

Il documento si articola in tre sezioni principali:

1. **Documento di analisi e specifica dei requisiti:**

Tale documento costituisce l'output finale del processo di ingegneria dei requisiti, strutturato in due macro-fasi: l'analisi, finalizzata alla rilevazione delle esigenze degli stakeholder, seguita da una fase di specifica, dedicata alla formalizzazione dei requisiti mediante descizioni testuali.

2. **Documento di design:**

Il presente documento si pone l'obiettivo di definire i criteri di progettazione del sistema, delineandone l'architettura complessiva, le scelte tecnologiche di riferimento e l'articolazione in sottosistemi. Il tutto affiancato da chiare rappresentazioni diagrammatiche.

3. **Attività di testing:**

Fase del software, volta ad individuazione anomalie e bugs. Tale attività è finalizzata a "stressare" i metodi pubblici di una classe, con lo scopo di validarne il corretto comportamento e a garantire la conformità dei componenti rispetto ai requisiti definiti.

Parte I

Documento di Analisi e Specifica dei Requisiti

Capitolo 1

Requisiti funzionali

1.1 Definizione dei casi d'uso

Attore	Casi d'uso
Admin	Creare nuove utenze
	Creare nuovo progetto
	Modificare un progetto
User	Effettuare login tramite email
	Visualizzare lista riepilogativa dei progetti
	Visualizzare lista riepilogativa delle issue
	Cercare una issue tramite tag
	Segnalare una issue
	Aggiungere un commento ad una issue
	Aggiornare lo stato di una issue

Tabella 1.1: Tabella dei casi d'uso

1.2 Descrizione dettagliata dei casi d'uso

- **Creare nuove utenze:** Gli utenti con privilegi di amministratore sono abilitati alla creazione di nuovi profili, definendone le credenziali (email e password) e assegnando il

relativo livello di autorizzazione ("Utente Standard" o "Amministratore").

- **Effettuare login tramite email:** L'accesso alla piattaforma è subordinato a una procedura di autenticazione, che richiede l'inserimento di credenziali valide (indirizzo email e password) preventivamente configurate dall'amministratore in fase di registrazione dell'utenza. In fase di primo avvio del sistema, l'accesso iniziale da parte del primo amministratore avverrà tramite credenziali predefinite, fornite dal team di sviluppo e già inserite all'interno del database.
- **Creare un nuovo progetto:** Il sistema consente agli amministratori di istanziare nuovi progetti, specificandone il titolo e definendo l'elenco degli utenti autorizzati a parteciparvi.
- **Modificare un progetto:** L'amministratore dispone dei permessi per modificare le proprietà di un progetto esistente, potendone aggiornare il titolo e gestire la composizione del team di lavoro tramite l'aggiunta o la rimozione di partecipanti.
- **Visualizzare lista riepilogativa progetti:** Il sistema fornisce a ciascun utente autenticato l'accesso a un elenco riepilogativo contenente unicamente i progetti a cui è stato formalmente assegnato.
- **Visualizzare lista riepilogativa delle issue:** Gli utenti autorizzati ad accedere a un progetto hanno la facoltà di consultare l'elenco completo delle segnalazioni (issue) ad esso correlate.
- **Filtrare/Ordinare lista riepilogativa delle issue:** Ogni utente può ordinare l'elenco delle issue in ordine cronologico (crescente o decrescente) in base alla data di creazione. È inoltre possibile applicare filtri combinati in base alla tipologia, alla priorità e allo stato della segnalazione.
- **Cercare issue tramite tag:** Il sistema prevede una funzionalità di ricerca che permette agli utenti di individuare specifiche segnalazioni avvalendosi delle etichette (tag) assegnate dal creatore durante la fase di apertura.
- **Segnalare una issue:** Tutti gli utenti autenticati hanno la possibilità di segnalare una issue indicandone un titolo e una descrizione. E' possibile, inoltre, che gli utenti vogliano specificare la priorità, il tipo e/o inserire un' immagine.
- **Aggiungere un commento ad una issue:** È garantita a tutti gli utenti la possibilità di interagire con le issue presenti nei propri progetti di competenza, contribuendo alla risoluzione mediante l'inserimento di commenti testuali, indipendentemente da chi abbia originato la segnalazione.
- **Aggiornare lo stato di una issue:** Il sistema autorizza la transizione di stato di una issue, ma tale aggiornamento è consentito esclusivamente all'autore della segnalazione.

1.3 Modellazione dei casi d'uso

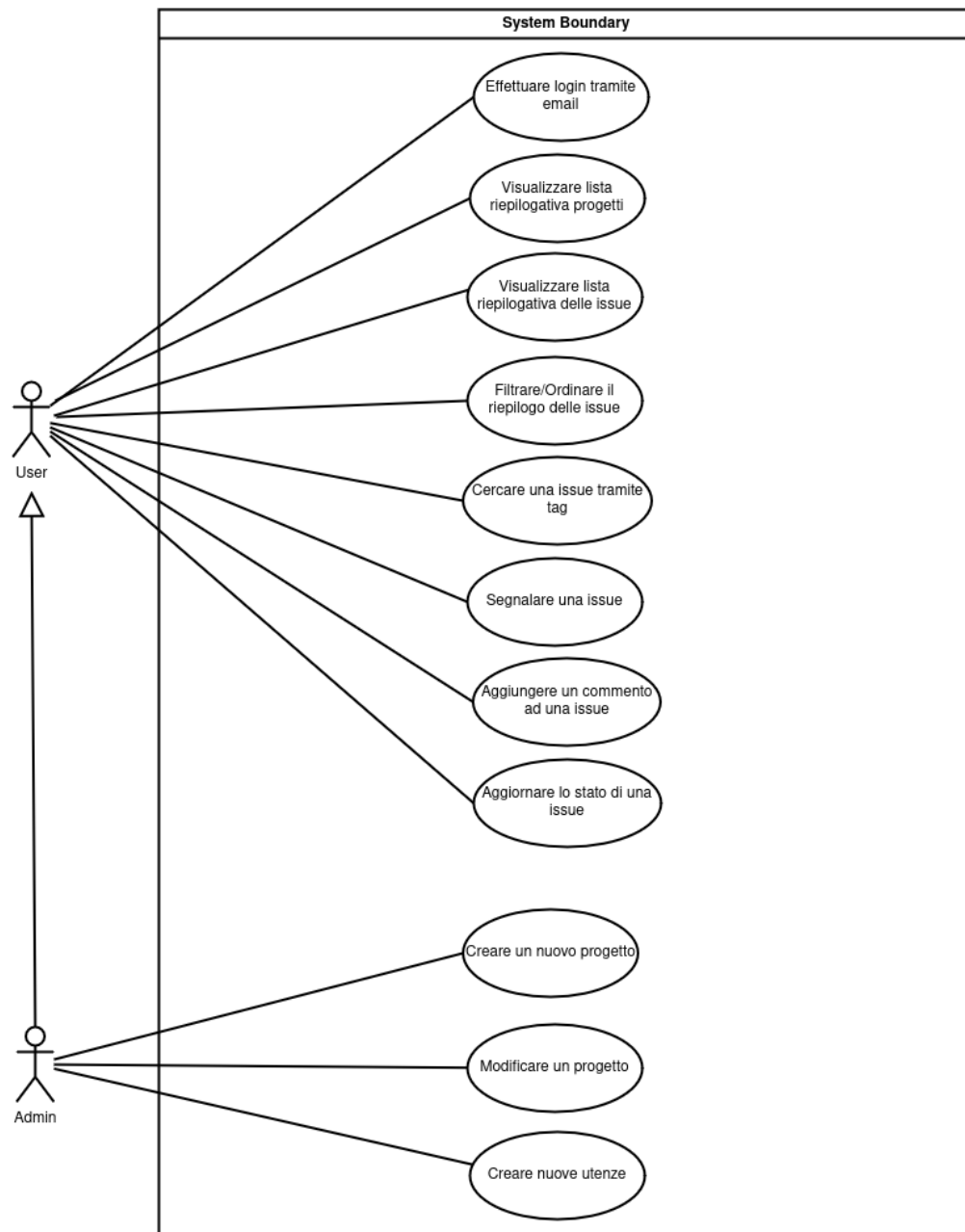


Figura 1.1: Diagramma dei casi d'uso

1.4 Formalizzazione casi d'uso

In questa sezione verranno analizzati nel dettaglio alcuni casi d'uso, descrivendo i flussi di interazione tra l'utente e il sistema. Per la loro rappresentazione si è scelto di adottare il formalismo strutturato proposto da Alistair Cockburn.

1.4.1 Segnalare una issue

Use case	Segnalare una issue		
Goal in Context	L'utente, in qualità di membro attivo di un progetto, intende registrare nel sistema una nuova segnalazione (issue).		
Preconditions	L'utente è regolarmente autenticato all'interno del sistema ed è formalmente associato come partecipante al progetto all'interno del quale intende aprire la segnalazione.		
Success end condition	Il sistema registra la issue segnalata dall'utente		
Failed end condition	Il sistema non registra la issue segnalata dall'utente		
Primary actor	Utente		
Trigger	L'utente clicca "New Issue" in M0		
Main scenario	Step	Utente	Sistema
	1	Clicca "New Issue" in M0	
	2		Mostra M1
	3	Compila i campi del form	
	4	Clicca su "Create Issue"	

	5		Mostra messaggio di successo e torna in M0
Extension n°1	Step	Utente	Sistema
	3.1	Non inserisce uno dei campi (eccetto immagine: è opzionale)	
	3.2	Clicca su “Create Issue”	
	3.3		Mostra messaggio di errore in base al campo che ha dimenticato
Extension n°2	Step	Utente	Sistema
	4.1	Clicca “Annulla”	
	4.3		Mostra M0

1.4.2 Modifica di un progetto

Use case	Modifica un progetto		
Goal in Context	L'amministratore intende aggiornare le proprietà di un progetto (titolo, partecipanti)		
Preconditions	L'utente ha effettuato l'accesso al sistema ed è in possesso dei privilegi di "Amministratore".		
Success end condition	Il sistema convalida i nuovi dati e aggiorna con successo il record del progetto nel database.		
Failed end condition	La validazione dei dati fallisce. I dati originari del progetto rimangono inalterati.		
Primary actor	Utente		
Trigger	L'utente clicca il bottone di modifica in M0		
Main scenario	Step	Utente	Sistema
	1	Seleziona l'opzione di modifica per uno specifico progetto in M0	
	2		Mostra M1
	3	Apporta le variazioni desiderate ai parametri e procede col salvataggio cliccando su "Save Changes"	
	4		Notifica l'avvenuto salvataggio e reindirizza l'utente alla schermata aggiornata in M0
Extension n°1	Step	Utente	Sistema

	2.1	Non inserisce informazioni e clicca "Annulla"	
	2.2		Mostra M0
Extension n°2	Step		Sistema
	3.1	Inserisce un indirizzo email non registrato a sistema per l'aggiunta di un partecipante	
	3.2		Notifica l'errore riscontrato e annulla l'associazione dell'utente al progetto

1.4.3 Aggiungere un commento ad una issue

Use case	Aggiungere un commento ad una issue		
Goal in Context	L'utente intende inserire un commento all'interno di una specifica issue		
Preconditions	L'utente è regolarmente autenticato all'interno del sistema ed è iscritto come partecipante al progetto al quale appartiene la issue. Il sistema ha caricato correttamente la pagina di dettaglio della issue in questione.		
Success end condition	Il commento viene inserito con successo nel sistema		
Failed end condition	Il commento non viene salvato correttamente		
Primary actor	Utente		
Trigger	L'utente seleziona la issue desiderata dall'elenco in M0		
Main scenario	Step	Utente	Sistema
	1	Seleziona una issue in M0	
	2		Mostra M1 con i dettagli della issue
	3	Inserisce un commento nel campo apposito e clicca "Invio"	
	4		Mostra il commento come ultimo commento
Extension n°1	Step	Utente	Sistema

	2.1	Mostra M1 con i dettagli della issue, rilevando che la issue si trova nello stato "closed"	
	2.2		Il sistema disabilita la funzionalità di inserimento commenti per la issue corrente.

1.5 Prototipazione Visuale: MockUp

Al fine di rendere più chiara la comprensione dei requisiti e fornire un'anteprima dell'applicativo, sono stati sviluppati dei prototipi semplici ed efficaci attraverso Figma.

1.5.1 Modifica di un progetto

L'utente amministratore decide di procedere alla modifica del titolo di un progetto specifico, attualmente registrato come "Travel App", aggiornandolo alla nuova denominazione "PlanYourTravel".

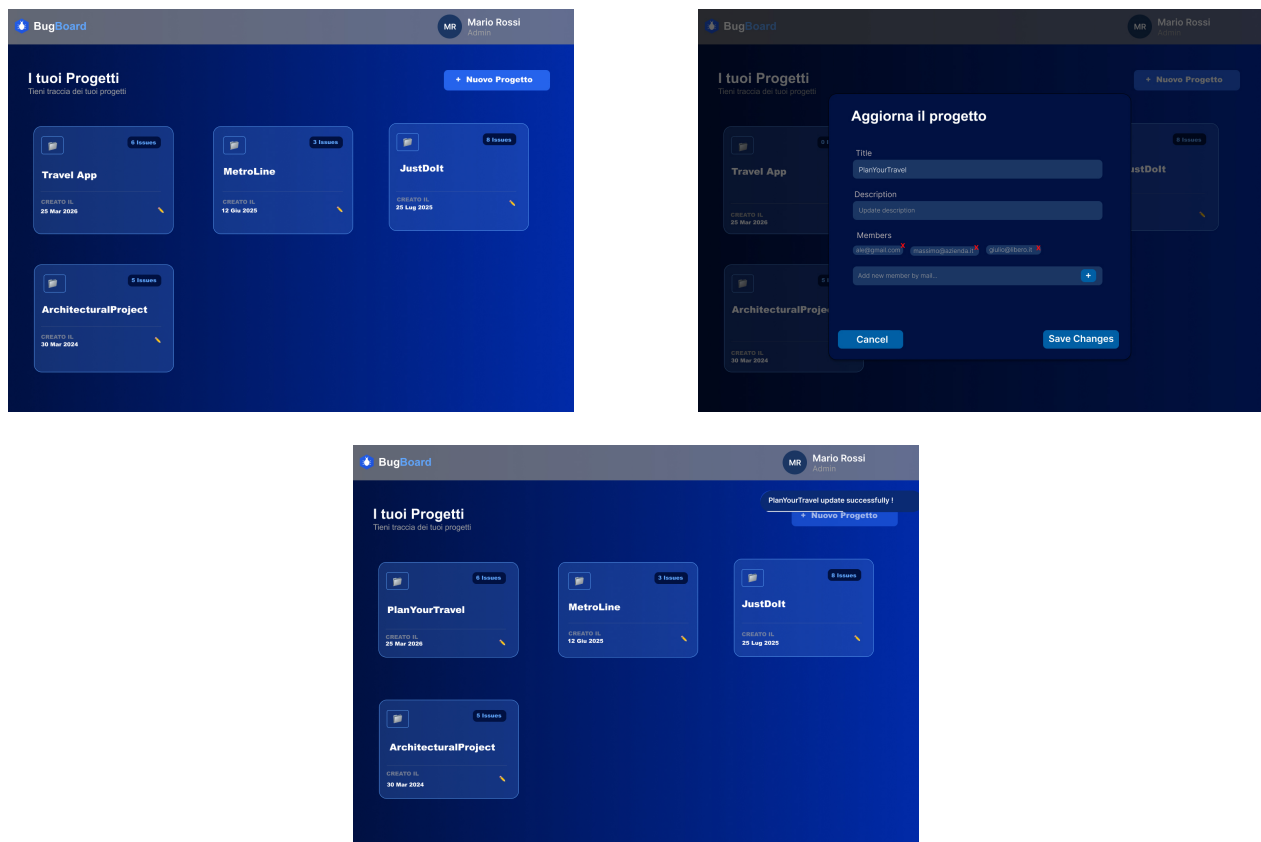


Figura 1.2: MockUp_01 Project Update

1 Attivazione della Modifica

L'utente amministratore, all'interno della schermata principale (Dashboard), individua il progetto target e interagisce con il pulsante di modifica specifico per quel record.

2 Input Dati e Salvataggio

Il sistema visualizza un componente di interfaccia di tipo *modal*. L'utente modifica il campo di testo relativo al titolo inserendo la nuova stringa "PlanYourTravel". Una volta completato l'inserimento, l'utente conferma l'operazione cliccando sul pulsante **Save Changes**.

3 Aggiornamento e Chiusura

Il sistema processa la richiesta, aggiorna il database e chiude automaticamente la finestra modale. L'utente viene riportato alla vista principale, dove vi sarà la visualizzazione di un messaggio di conferma per notificare il successo dell'operazione. Il titolo del progetto risulta aggiornato in tempo reale.

1.5.2 Segnalare una issue

L'utente decide di inserire una nuova issue associata al progetto **MetroLine**

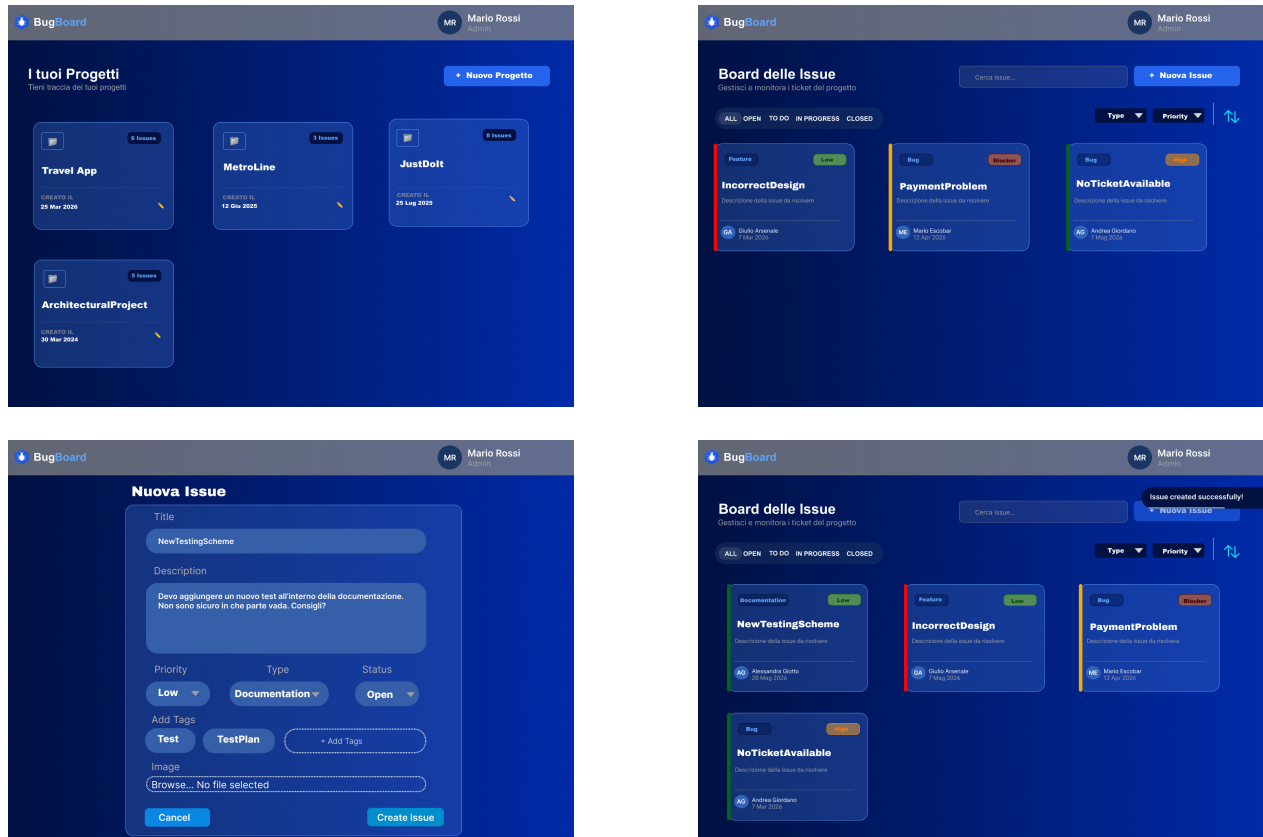


Figura 1.3: MockUp_02 NewIssue

1 Navigazione al Modulo Issue

L'utente seleziona il progetto di riferimento dalla dashboard, in questo caso "Metro-Line" e naviga verso la pagina dedicata alla gestione delle segnalazioni.

2 Inizializzazione della Creazione

All'interno della pagina delle issue, l'utente clicca sul pulsante di azione "Nuova Issue" per aprire il modulo di inserimento dati.

3 Compilazione dei Dati e Salvataggio

Il sistema presenta un form vuoto. L'utente inserisce le informazioni nei campi *Titolo* e *Descrizione*, associa i tag [test] e [testplan] e decide di non allegare

immagini. Seleziona una priorità `Low`, un Tipo `Documentation` e uno status `Open`. L'utente conferma l'operazione cliccando sul pulsante **Salva**.

4 Aggiornamento e Conclusione

Il sistema processa la richiesta, aggiorna il database, visualizza un messaggio di successo e reindirizza automaticamente l'utente alla pagina dell'elenco issue. La vista viene aggiornata in tempo reale e la nuova segnalazione risulta immediatamente visibile in cima alla lista.

1.5.3 Aggiungere un commento ad una issue

L'utente **Mario Rossi** decide di scrivere un commento con lo scopo di contribuire nella risoluzione della issue

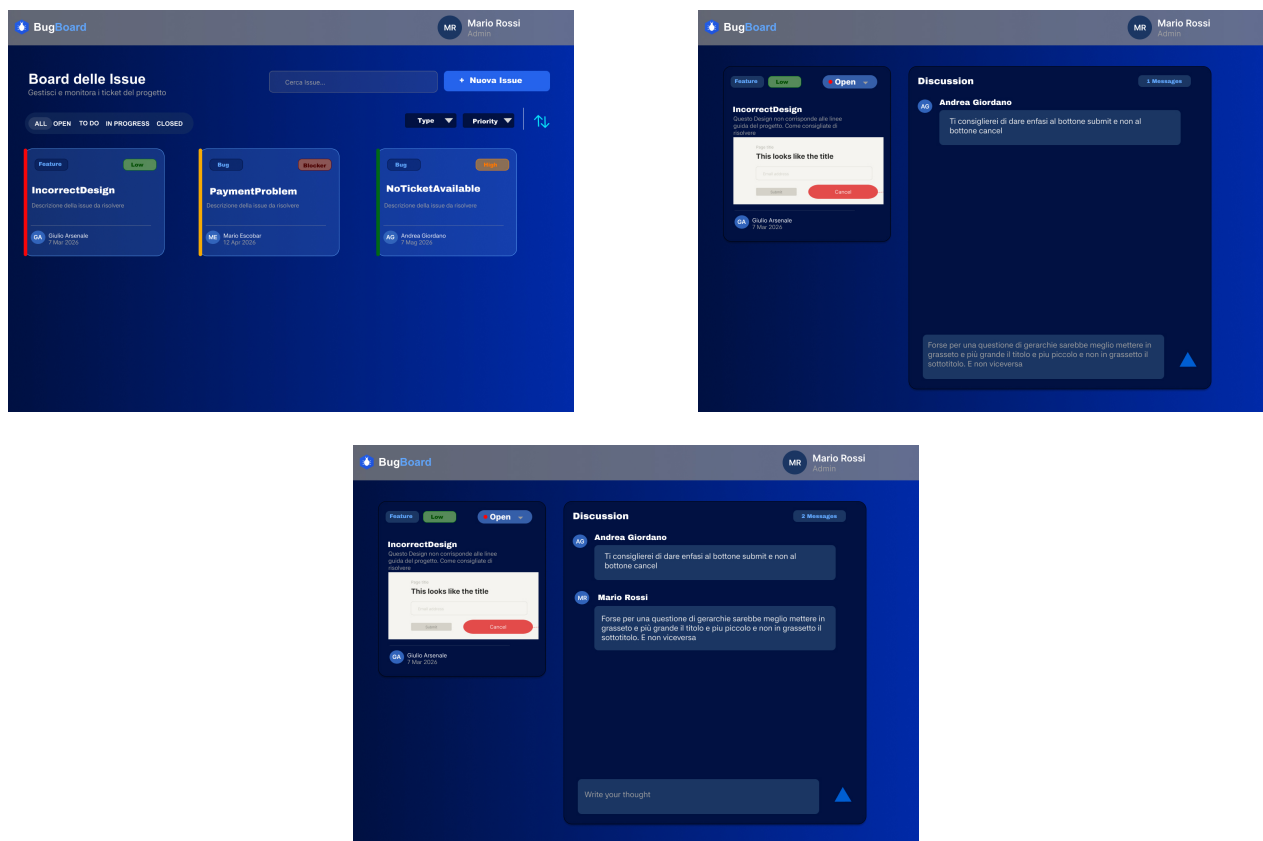


Figura 1.4: MockUp_03 AddNewComment

1 Navigazione alla pagina di gestione della issue

L'utente seleziona la issue in cui vuole inserire un commento.

2 All'interno della pagina di gestione della issue, l'utente clicca sul campo **Write your thought** per inserire il suo commento, poi procede con l'invio.

3 Salvataggio

Il sistema inserisce il commento nella sezione di Discussione e appare come nuovo commento

Capitolo 2

Requisiti non funzionali

2.1 Usabilità

- **RNF-01:** L'interfaccia di login deve essere intuitiva e il sistema di autenticazione deve permettere a un utente registrato di accedere alla dashboard entro 3 interazioni(click,touch).
- **RNF-02:** Il sistema deve fornire un'interfaccia utente chiara e coerente, tale da consentire all'utente di comprendere facilmente le funzionalità disponibili e le azioni eseguibili.
- **RNF-03:** Il sistema deve fornire un feedback immediato e comprensibile a seguito delle azioni dell'utente, indicando l'esito delle operazioni effettuate.

2.2 Sicurezza

- **RNF-04:** Il sistema deve proteggere le credenziali tramite algoritmi di hashing moderni e robusti. Le password degli utenti devono quindi essere memorizzate in forma cifrata e non in chiaro.
- **RNF-05:** La password deve essere robusta e rispettare i requisiti minimi di sicurezza: lunghezza minima di 8 caratteri di cui almeno una maiuscola, una minuscola, un numero e un carattere speciale.
- **RNF-06:** Il sistema deve garantire che i dati siano rigorosamente validati sia lato client che lato server per prevenire l'inserimento di dati inconsistenti o attacchi XSS/SQL Injection.
- **RNF-07:** Il backend deve essere l'unica autorità per la validazione e il salvataggio dei dati. Tutte le operazioni CRUD sui dati persistenti devono essere eseguite esclusivamente tramite backend, impedendo manipolazioni dirette dal client.
- **RNF-08:** L'Access Token, utilizzato per l'accesso alle risorse protette, deve avere una validità massima limitata a 5 minuti.
- **RNF-10:** Il Refresh Token, utilizzato esclusivamente per richiedere un nuovo access token quando il precedente scade, deve avere una validità massima di 7 giorni.

- **RNF-09:** Il sistema deve validare il formato dell'email tramite espressioni regolari conformi allo standard RFC 5322. Inoltre deve trattare gli indirizzi email come case insensitive per evitare duplicati accidentali.

2.3 Portabilità

- **RNF-10:** Il sistema deve fornire un'interfaccia web responsive in grado di adattarsi a diverse risoluzioni e dispositivi (desktop, mobile, tablet), garantendo accesso completo alle funzionalità

Capitolo 3

Requisiti di dominio

- **RD-01 Definizione dei ruoli dell'utente:** Il dominio applicativo prevede due ruoli principali:
 - **Admin:** Dipendenti dell'azienda avente privilegi da amministratore.
 - **User:** Dipendenti dell'azienda che si occupano dello svolgimento dei task.

Le funzionalità e le operazioni devono essere regolate in base al ruolo associato all'utente autenticato.

- **RD-02 Categorizzazione delle issue:** Ogni issue deve essere classificata secondo criteri di categorizzazione basati sulla priorità, con lo scopo di determinarne il livello di urgenza.
- **RD-03 Ciclo di vita di una issue:** Ogni issue deve essere gestita attraverso un ciclo di vita definito da stati e transizioni, che ne determinano i possibili avanzamenti e cambi di stato secondo un flusso di lavoro predefinito.

Capitolo 4

Target Utenti

Durante la progettazione di un nuovo software, è fondamentale individuare e definire il target degli utenti coinvolti nell'uso dell'applicazione.

La fase di elicitazione e analisi dei requisiti ha come obiettivo primario l'individuazione delle necessità e delle aspettative dei committenti e degli utenti finali.

4.1 Personas

Per avere una visione più chiara sul target, è stata realizzata una breve indagine volta alla determinazione di possibili profili di utenti finali.

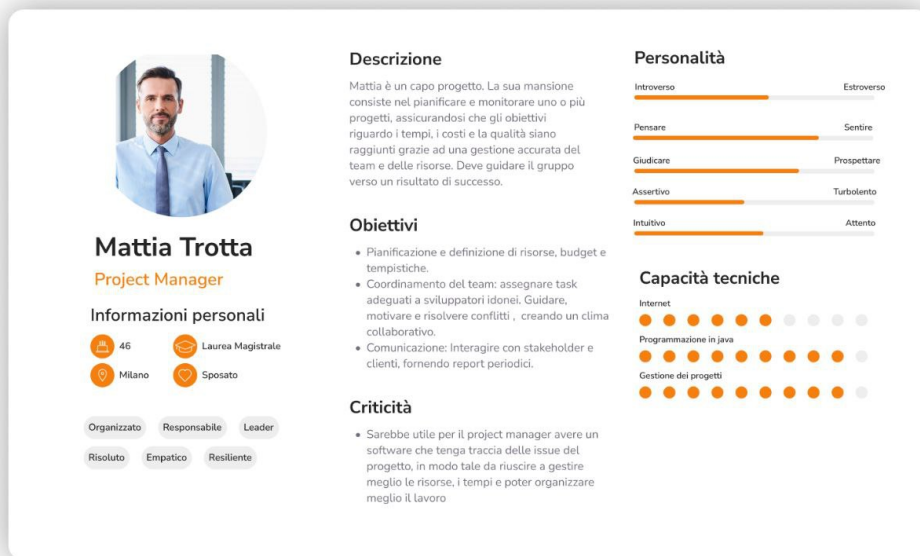


Figura 4.1: Personas_01 Project Manger

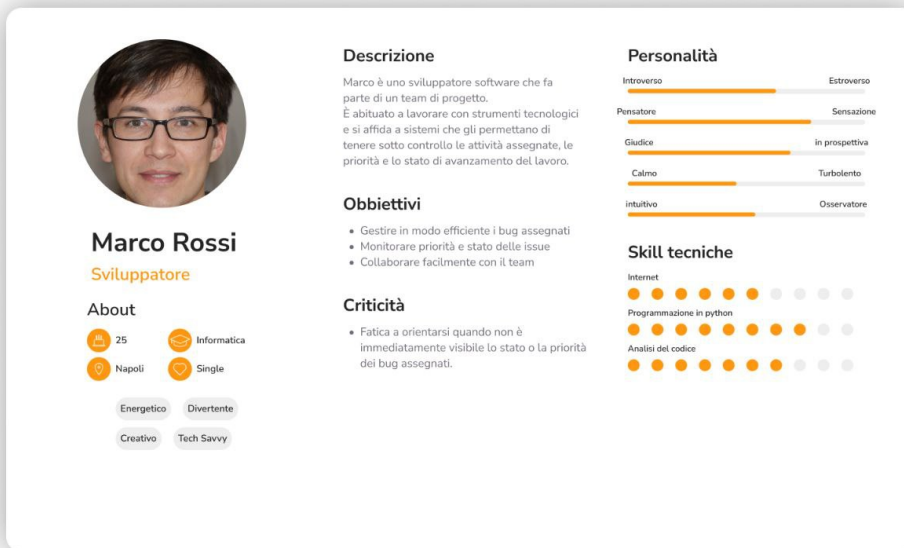


Figura 4.2: Personas_02 Developer

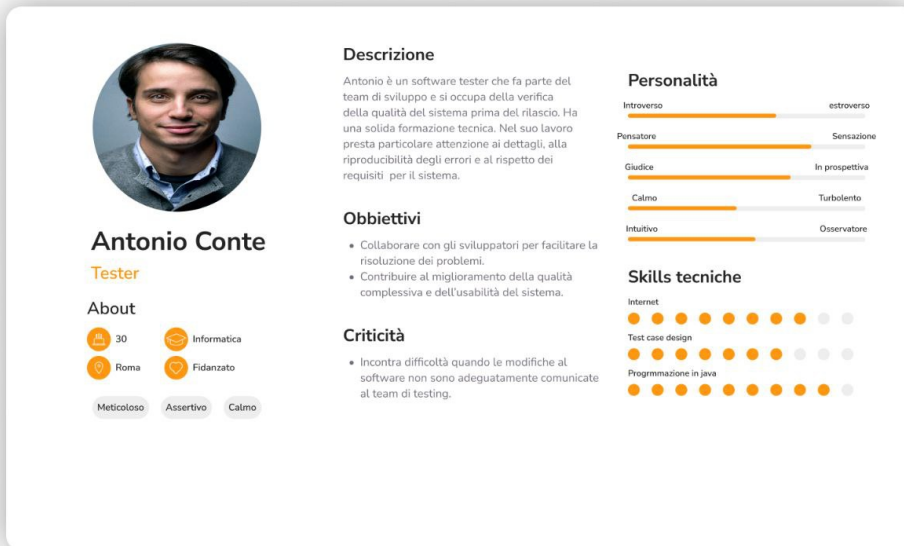


Figura 4.3: Personas_03 Tester

4.1.1 Tratti caratteristici

Dall'analisi delle Personas emergono profili con obiettivi e modalità d'uso della piattaforma distinti. Di seguito vengono delineati i tratti salienti per ogni categoria:

- **Project Manager:** Utilizza la piattaforma come strumento di monitoraggio. Il suo interesse principale non è la singola riga del codice, ma il progetto nella sua complessità. Il suo obiettivo è supervisionare l'avanzamento dei lavori, rispettare le scadenze e distribuire il carico di lavoro.
Fa uso della piattaforma per consultare la dashboard e i report.
- **Developer:** Rappresenta colui che vive la piattaforma come uno strumento operativo quotidiano. La sua priorità è l'efficienza nella risoluzione dei problemi e la chiarezza delle informazioni tecniche.
Il suo obiettivo principale è identificare i bug, documentarli, contribuire nella risoluzione.
- **Tester:** Il tester, è colui che ispeziona il sistema e ne garantisce la qualità (software quality).
Ha l'obiettivo di individuare falle nel sistema, verificare le correzioni degli sviluppatori e garantire che il prodotto rispetti i requisiti.

Parte II

Documento di Design

Capitolo 5

Architettura del sistema

5.1 Analisi dell'architettura

Il sistema realizzato adotta uno stile architetturale *Client-Server* disaccoppiato, tipico delle moderne applicazioni web.

Il *Client* rappresenta il livello di presentazione del sistema ed è responsabile della visualizzazione dell'interfaccia grafica e della gestione dell'interazione con l'utente. Il *Server*, invece, concentra la logica applicativa principale, la gestione della persistenza e l'elaborazione dei dati.

L'interazione tra client e server avviene tramite richieste asincrone basate sul protocollo HTTP/HTTPS. Lo scambio delle informazioni è realizzato mediante payload in formato JSON, favorendo il disaccoppiamento tra frontend e backend e rendendo più chiara la separazione delle responsabilità tra i diversi livelli del sistema.

5.1.1 Client



Il client è sviluppato come una **SPA (Single Page Application)** utilizzando il framework **Angular**. L'architettura del frontend si basa su un approccio **Component-Based**, il quale mappa i principi del Pattern **MVVM (Model-View-ViewModel)** secondo la seguente suddivisione:

- **View**: Rappresentata dai template HTML dei componenti Angular, deputati esclusivamente alla resa grafica.
- **ViewModel**: Costituito dalle classi Typescript dei componenti stessi, che gestiscono lo stato locale dell'interfaccia e catturano gli eventi utente.
- **Model**: Rappresentano delle interfacce di tipizzazione (interfaces) che definiscono la struttura dei dati e del dominio applicativo.

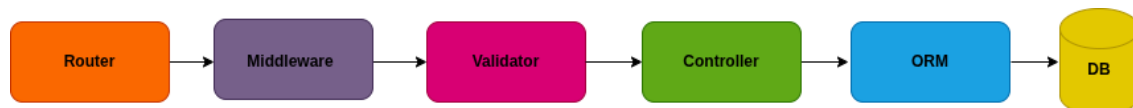
L'adozione di questa struttura offre i seguenti vantaggi:

- **Separazione delle responsabilità e basso Accoppiamento:** Isolando la logica di presentazione (ViewModel) dalla visualizzazione pura (View) si riduce la propagazione di effetti collaterali a seguito di modifiche al codice.
- **Modularità e Riutilizzabilità del codice:** La suddivisione dell'applicazione in componenti e servizi con responsabilità ben definite favorisce il riutilizzo delle funzionalità in differenti contesti applicativi, riducendo la duplicazione del codice e migliorando la modularità complessiva del sistema.
- **Testabilità:** La chiara distinzione tra View e ViewModel consente di isolare la logica dell'interfaccia utente dal livello di presentazione grafica, rendendo possibile l'esecuzione di test unitari sulla logica applicativa senza la necessità di coinvolgere l'interfaccia utente o il contesto di rendering.

A completamento dell'architettura del Client, vengono sfruttati tre costrutti fondamentali del framework:

- **Services:** La logica di comunicazione con il backend è delegata a servizi dedicati, che incapsulano il modulo *HttpClient*. I componenti non interrogano mai direttamente la rete, ma interagiscono con i servizi, preservando la separazione tra i livelli.
- **Routing Guards:** Strumenti dedicati al controllo degli accessi sulle rotte del frontend, che impediscono la navigazione verso componenti protetti se non sono soddisfatti i requisiti di autenticazione.
- **Dependency injection:** Meccanismo fornito da Angular che gestisce le dipendenze tra componenti e servizi, migliorando manutenibilità del codice.

5.1.2 Server



Il backend è sviluppato in ambiente **Node.js**, utilizzando il framework **Express** progettato per la creazione di servizi web e API RESTful.

Il server adotta una **Layered Architecture** che prevede una chiara separazione delle responsabilità tra i diversi componenti del sistema attraverso i seguenti strati:

- **Livello di Routing:** Ha l'unico compito di definire gli endpoints URL e indirizzare le richieste HTTP entranti verso i corretti controller.
- **Livello Middleware:** Gestiscono logiche trasversali come autenticazione e logging prima che la richiesta arrivi a destinazione. Si occupano di controllare che chi sta effettuando la richiesta ha le autorizzazioni per effettuarla.
- **Livello di Validazione:** Strato intermedio che intercetta i dati in ingresso e ne verifica la conformità rispetto a schemi predefiniti

- **Livello di Controller:** Costituisce il nucleo della logica id business. Riceve i dati validati, coordina le operazioni necessarie interagendo con il livello di persistenza ed elabora la risposta per il client.
- **Livello di Persistenza:** Gestito tramite l'ORM **Sequelize**, il quale mappa le tabelle del database relazionale **PostgreSQL** in oggetti JavaScript, isolando l'applicazione dalla sintassi SQL cruda.

Le funzionalità trasversali, quali validazione delle richieste, autenticazione tramite token , autorizzazione e gestione dei file, sono delegate a middleware dedicati.

Questa organizzazione favorisce modularità, manutenibilità e testabilità, garantendo una chiara separazione delle responsabilità.

L'architettura può essere ricondotta a una variante del pattern **MVC (Model-View-Controller)** applicato ai servizi Web, in cui il ruolo della View è sostituito dalla rappresentazione delle risorse tramite risposte in formato Json.

Capitolo 6

Tecnologie e strumenti di sviluppo

L'architettura software dell'applicazione è stata progettata seguendo criteri di scalabilità, manutenibilità e robustezza. Di seguito vengono analizzate in dettaglio le tecnologie, i framework e gli strumenti infrastrutturali adottati per la realizzazione dell'intero ecosistema software.

6.1 Architettura Backend e Persistenza dei Dati

6.1.1 Node.js & Express

Node.js è un ambiente di runtime JavaScript, utilizzato in combinazione con il Framework **Express**. Express funge da middleware per la gestione del routing delle API RESTful, la manipolazione delle richieste HTTP in ingresso e la strutturazione delle risposte JSON inviate al client. La combinazione di queste due tecnologie permette una rapida implementazione dei servizi di backend e una gestione pulita dei flussi logici.

6.1.2 PostgreSQL & Sequelize

Per la persistenza dei dati è stato adottato **PostgreSQL**, un sistema di gestione di database relazionali (RDBMS) open-source. PostgreSQL garantisce una forte aderenza agli standard ACID (Atomicity, Consistency, Isolation, Durability), supportando transazioni complesse, vincoli di integrità e query analitiche avanzate.

L'interazione tra il codice applicativo e il database non avviene tramite query SQL grezze, bensì attraverso **Sequelize**, un Object-Relational Mapping (ORM) per Node.js. Sequelize mappa le tabelle del database relazionale in oggetti della logica applicativa, astrando la gestione delle migrazioni dello schema, la validazione dei dati e la definizione delle relazioni (uno-a-molti, molti-a-molti), riducendo il rischio di vulnerabilità come le SQL Injection e velocizzando i tempi di sviluppo.

6.2 Sviluppo Frontend e Interfaccia Utente

La sezione client-side dell'applicazione è stata sviluppata per offrire un'esperienza utente (UX) reattiva, un'interfaccia modulare e componenti altamente riutilizzabili.

6.2.1 Angular

Angular è il framework di sviluppo open-source utilizzato. Basato su TypeScript permette di creare applicazioni web a pagina singola (SPA - Single Page Application). È stata scelta per la sua architettura solida che impone una struttura rigorosa che favorisce la manutenibilità del codice su larga scala, facilitando la separazione tra la logica di business e la vista.

6.2.2 Tailwind CSS

Per lo stile e l'aspetto visivo è stato integrato **Tailwind CSS**, un framework CSS. Questo approccio riduce drasticamente la dimensione dei file CSS finali e accelera la creazione di interfacce totalmente responsive e coerenti.

6.2.3 Toastr Notifications

Al fine di migliorare l'interazione con l'utente e fornire feedback immediati sulle operazioni asincrone (es. salvataggi completati, errori del server, avvisi), è stata integrata la libreria **Toastr**. Questa permette la visualizzazione di notifiche push non bloccanti (toast messages) stilizzate, temporizzate e configurabili.

6.3 Qualità del Codice e Testing Strutturato

La stabilità e la manutenibilità del software sono garantite dall'adozione di rigorosi standard di qualità del codice e da una suite di test che copre diversi livelli della piramide del testing.

6.3.1 SonarQube

La qualità e la sicurezza del codice sorgente sono monitorate costantemente tramite **SonarQube**, una piattaforma di analisi statica del codice. SonarQube analizza i repository evidenziando bug latenti, vulnerabilità di sicurezza, code smells (debiti tecnici) e duplicazioni di codice, garantendo il rispetto delle linee guida di programmazione pulita.

6.3.2 Jest Testing

Il testing unitario e di integrazione del backend è affidato a **Jest**, un framework di testing JavaScript focalizzato sulla semplicità e sulle prestazioni. Jest permette l'esecuzione rapida dei test in parallelo, la generazione automatica dei report di code coverage e fornisce potenti strumenti di mocking per isolare le singole funzioni o moduli dalle dipendenze esterne (come i servizi o il database), garantendo che ogni componente moduli funzioni esattamente come previsto.

6.3.3 Playwright

Per il testing End-to-End (E2E) è stato selezionato **Playwright**. Questo framework consente di automatizzare le interazioni sul browser simulando fedelmente il comportamento di un utente reale sull'interfaccia grafica.

6.4 Infrastruttura, DevOps e Ciclo di Vita del Software

La gestione del ciclo di vita del software, la containerizzazione e il deployment in cloud seguono le moderne metodologie DevOps per garantire rilasci rapidi e un'infrastruttura resiliente.

6.4.1 Docker

L'applicazione sfrutta **Docker** per la containerizzazione del backend. Tramite la definizione di Dockerfile e file di configurazione Docker Compose, l'ambiente operativo viene isolato all'interno di container, garantendo una parità assoluta tra l'ambiente di sviluppo locale, l'ambiente di test e quello di produzione.

Dockerfile

```
1 FROM node:22
2
3 WORKDIR /app
4
5 COPY package.json package-lock.json ./
6
7 RUN npm install
8
9 COPY . .
10
11 EXPOSE 3000
12
13 CMD ["npm", "run", "dev"]
```

Dockercompose

```
1 services:
2
3   backend:
4     build: .
5
6     container_name: bugboard-backend
7
8     ports:
```

```

9         - "3000:3000"
10
11     env_file:
12         - .env.local
13
14     volumes:
15         - ./app
16         - /app/node_modules
17         - uploads_data:/app/uploads
18
19     depends_on:
20         postgres:
21             condition: service_healthy
22
23     restart: unless-stopped
24
25 postgres:
26     image: postgres:17
27
28     container_name: bugboard-postgres
29
30     environment:
31         POSTGRES_DB: bugboard
32         POSTGRES_USER: postgres
33         POSTGRES_PASSWORD: postgres
34
35     ports:
36         - "5432:5432"
37
38     volumes:
39         - postgres_data:/var/lib/postgresql/data
40
41     healthcheck:
42         test: ["CMD-SHELL", "pg_isready -U postgres -d bugboard"]
43         interval: 5s
44         timeout: 5s
45         retries: 10
46
47     restart: unless-stopped
48
49 volumes:
50     postgres_data:
51     uploads_data:

```

6.4.2 Azure

Il cloud computing e l'hosting dell'intera infrastruttura applicativa sono affidati a **Microsoft Azure**. La piattaforma cloud fornisce i servizi necessari per ospitare i container Docker in produzione e garantire l'alta disponibilità del database e degli endpoint applicativi.

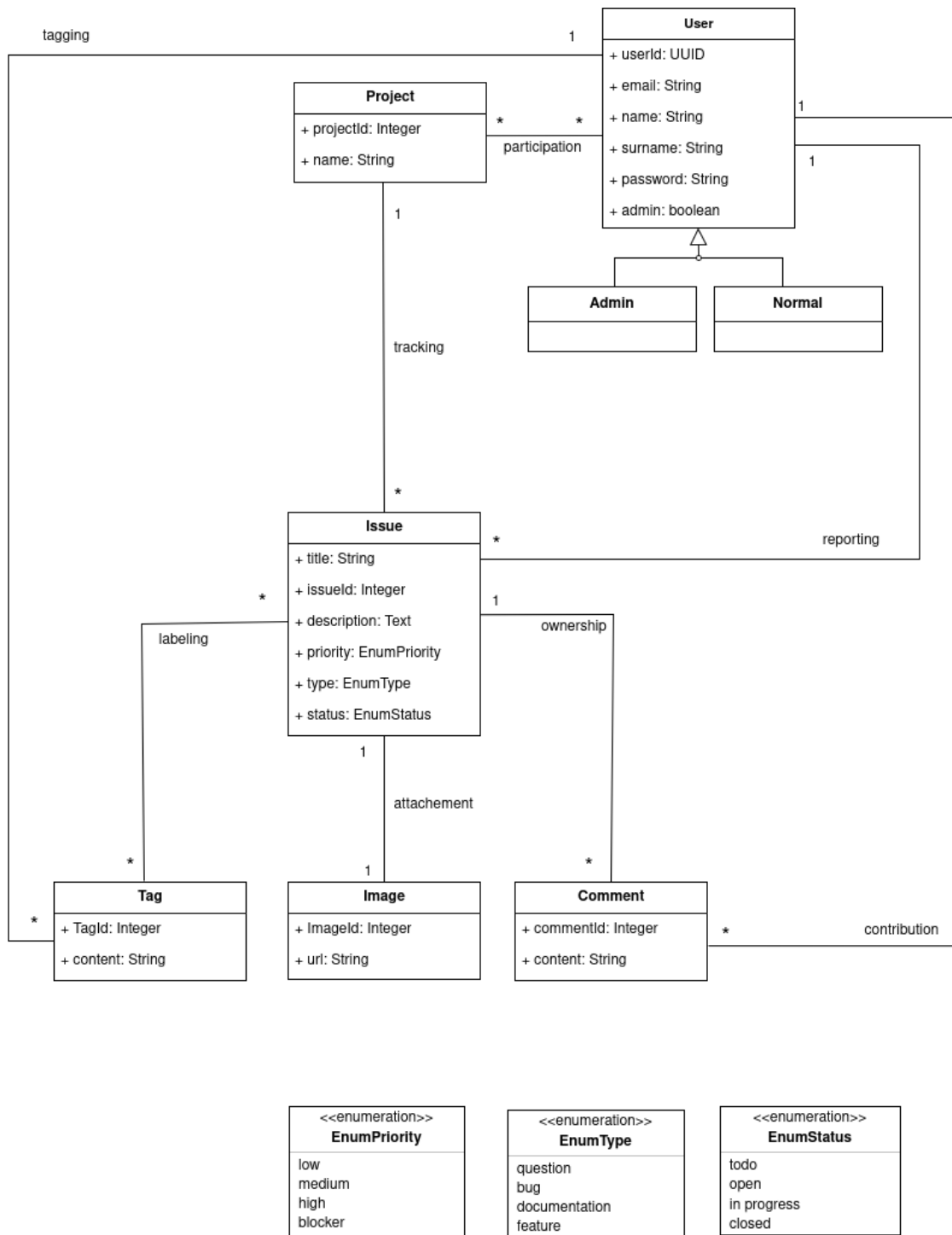
Capitolo 7

Persistenza dati

7.1 Descrizione dello schema per la persistenza dati

Al fine di comprendere e formalizzare la struttura logica della persistenza dei dati, si è scelto di realizzare un diagramma UML che specifichi ogni entità identificata nel sistema, dettagliando gli attributi principali e le relazioni con le altre componenti del dominio. Si procede con una breve descrizione per ogni entità e successivamente al diagramma vero e proprio.

- **User:** Rappresenta l'entità fondamentale per la gestione degli accessi, dell'autenticazione e del tracciamento delle responsabilità all'interno della piattaforma.
Il modello applica il principio di specializzazione dividendosi in due sottoclassi: **Admin** (per gli utenti con privilegi di gestione del sistema) e **Normal** (per gli utenti standard della piattaforma).
Relazioni principali: Partecipa a più progetti, può segnalare più Issue, creare tag e contribuire attivamente tramite commenti.
- **Project:** Rappresenta l'entità logica nella quale si organizza il lavoro. Funge da contenitore per le segnalazioni.
Relazioni principali: Un progetto può coinvolgere molteplici utenti e traccia al suo interno un numero indefinito di issue.
- **Issue:** È il nucleo operativo dell'applicazione. Rappresenta un bug, una feature o un'attività di documentazione da svolgere.
Relazioni principali: È al di sotto di un progetto. Può essere creata da un utente. Ad essa possono essere collegati molteplici commenti, molteplici tag e un'immagine rappresentativa del problema.
- **Comment:** Consente la collaborazione e l'interazione testuale tra i vari membri del team in merito ad una specifica problematica.
Relazioni principali: Un commento è legato al solo utente che lo scrive e alla sola issue specifica.
- **Tag:** Permette la ricerca personalizzata delle issue tramite parole chiave e un miglior filtraggio.
Relazioni principali: Un tag può essere utilizzato per più issue.



Capitolo 8

Diagrammi UML delle classi di design

8.1 Diagrammi delle classi di design

In questa sezione analizziamo, tramite l'uso di diagrammi UML, i requisiti funzionali identificati nella fase di analisi dei requisiti, con lo scopo di visualizzare le soluzioni adottate per ognuno di essi.

8.1.1 Autenticazione e gestione delle utenze

La sicurezza e l'accesso alla piattaforma sono regolati da politiche rigide di controllo degli accessi. A differenza delle comuni applicazioni web, il sistema non prevede una funzionalità di auto-registrazione da parte dell'utente.

Il ciclo di vita di un utenza e il relativo accesso sono governati dalle seguenti logiche:

- **Creazione delle utenze:** È una funzionalità esclusiva dell'amministratore. L'admin, autenticato sulla piattaforma, inserisce i dati del nuovo utente. La comunicazione delle credenziali avviene tramite comunicazione diretta. (Verrà presentata nella sezione successiva)
- **Autenticazione:** L'accesso all'applicazione è consentito a qualunque utente registrato e solo tramite l'inserimento dell'indirizzo email e della password associata. (E' presentato nella seguente sezione)

8.1.2 Autenticazione

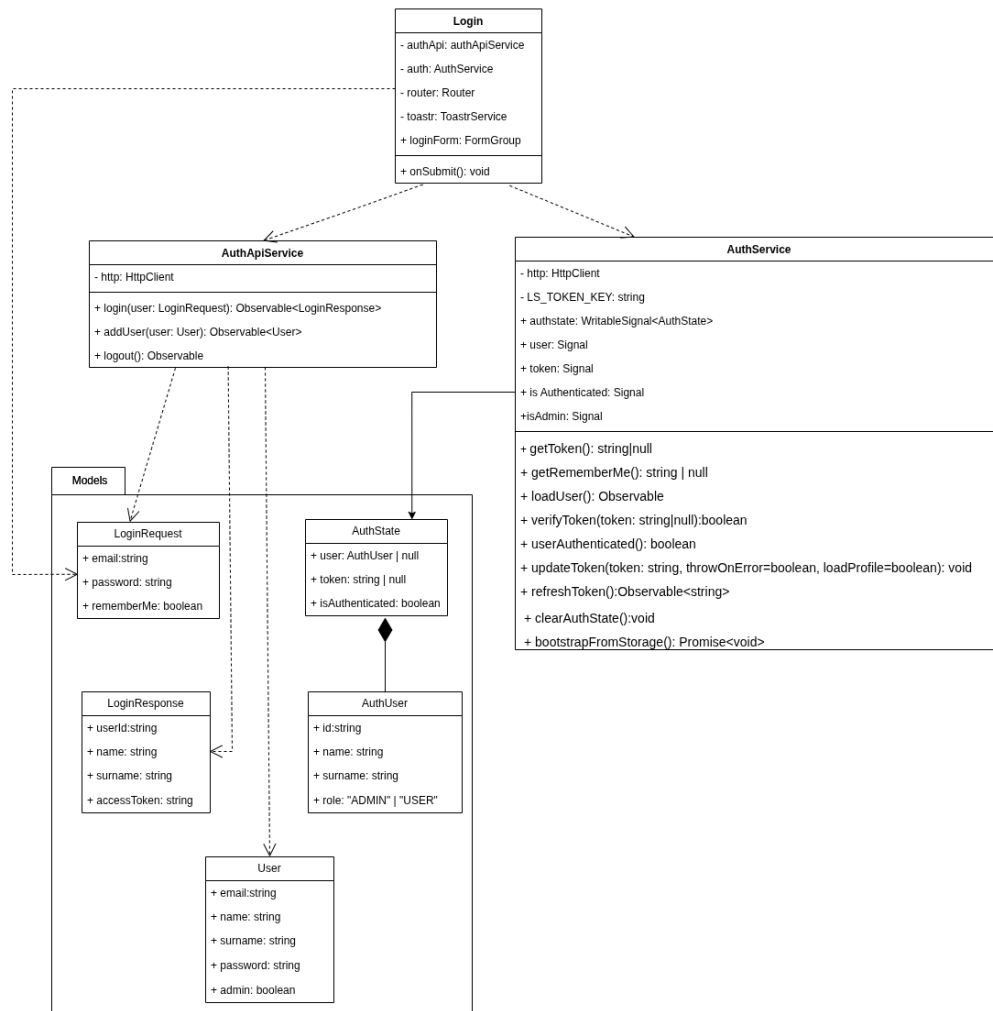


Figura 8.1: Diagramma delle classi del Login

8.2 Gestione dei progetti

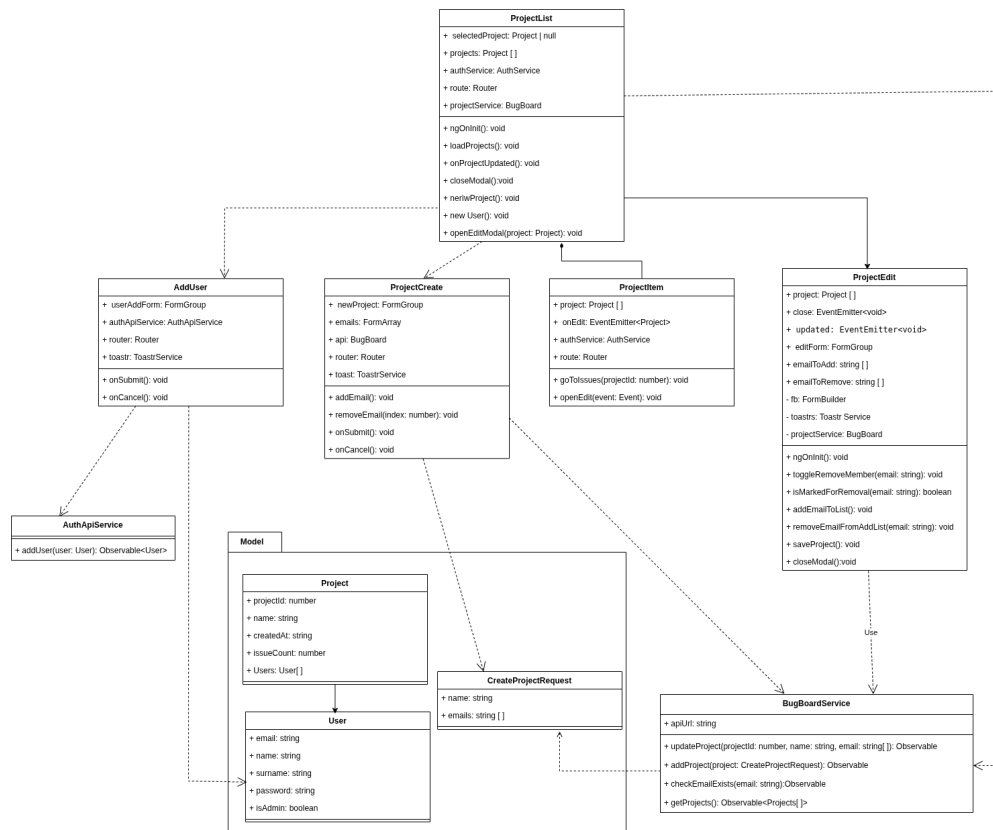


Figura 8.2: Diagramma delle classi per la gestione dei progetti

8.3 Gestione delle issue e commenti

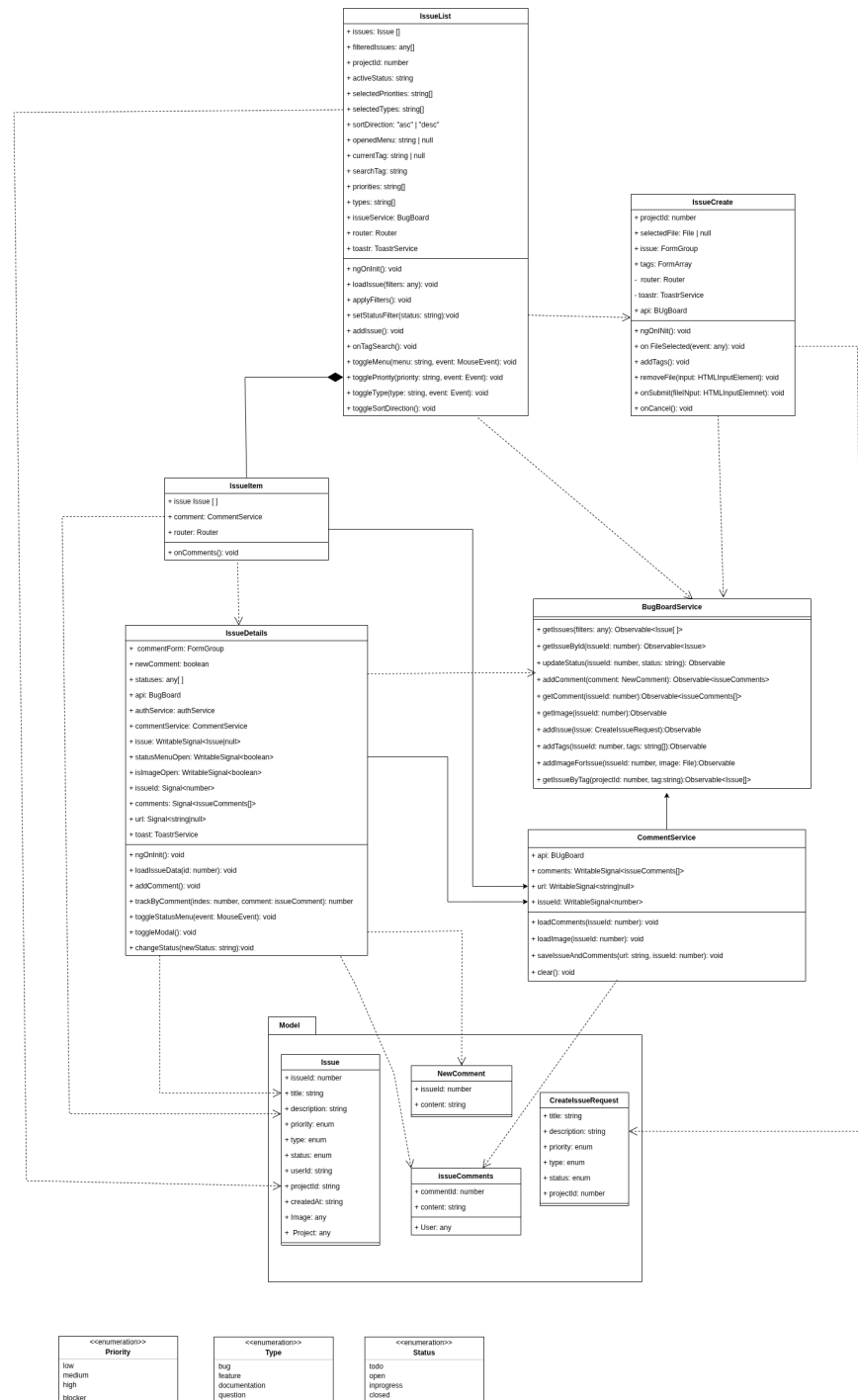


Figura 8.3: Diagramma delle classi per la gestione delle issue

8.4 Sequence Diagram

8.4.1 Creazione di una issue

Si è ritenuto opportuno inserire il seguente diagramma di sequenza, poichè rappresenta il caso d'uso più completo all'interno dell'applicazione, mostrando tutte le possibili interazioni tra frontend, servizi, backend e database.

La rappresentazione consente di identificare chiaramente i confini e le responsabilità dei vari componenti del sistema, distinguendo il ruolo dell'interfaccia utente (**issueCreate**), del servizio di comunicazione (**BugBoardService**), delle **API Backend** e del **Database**.

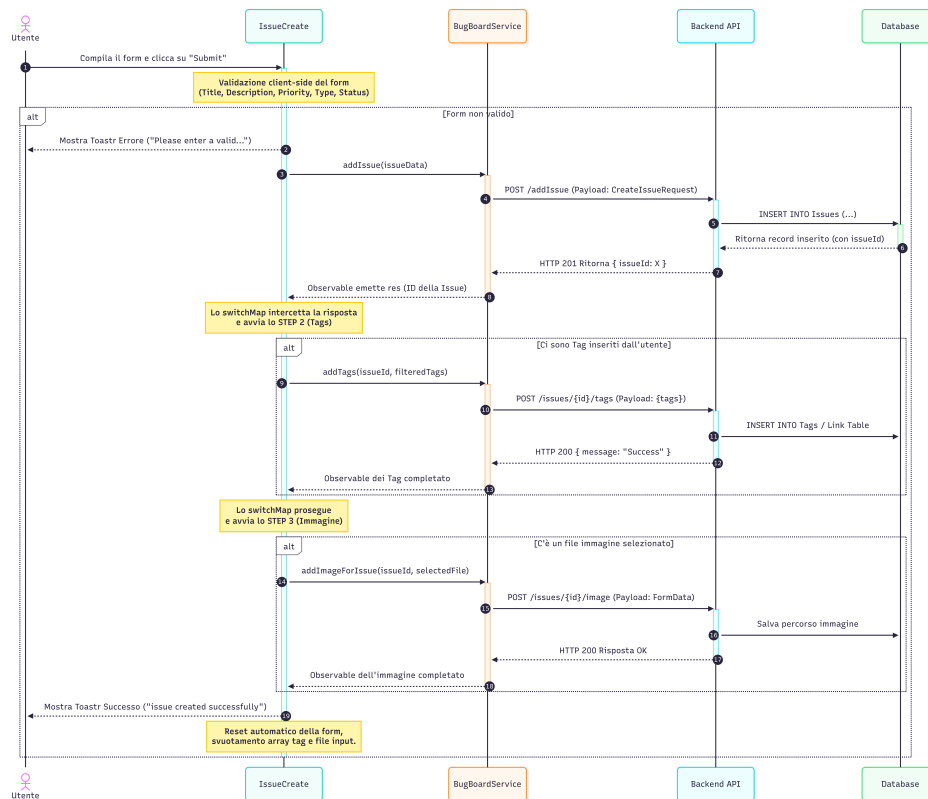


Figura 8.4: Diagramma di sequenza per la creazione di una issue

8.5 Statechart

8.5.1 Gestione dello stato di una issue

Si è ritenuto opportuno inserire il seguente diagramma poichè descrive in modo chiaro il ciclo di vita di una issue e le regole che governano le transizioni tra i diversi stati possibili.

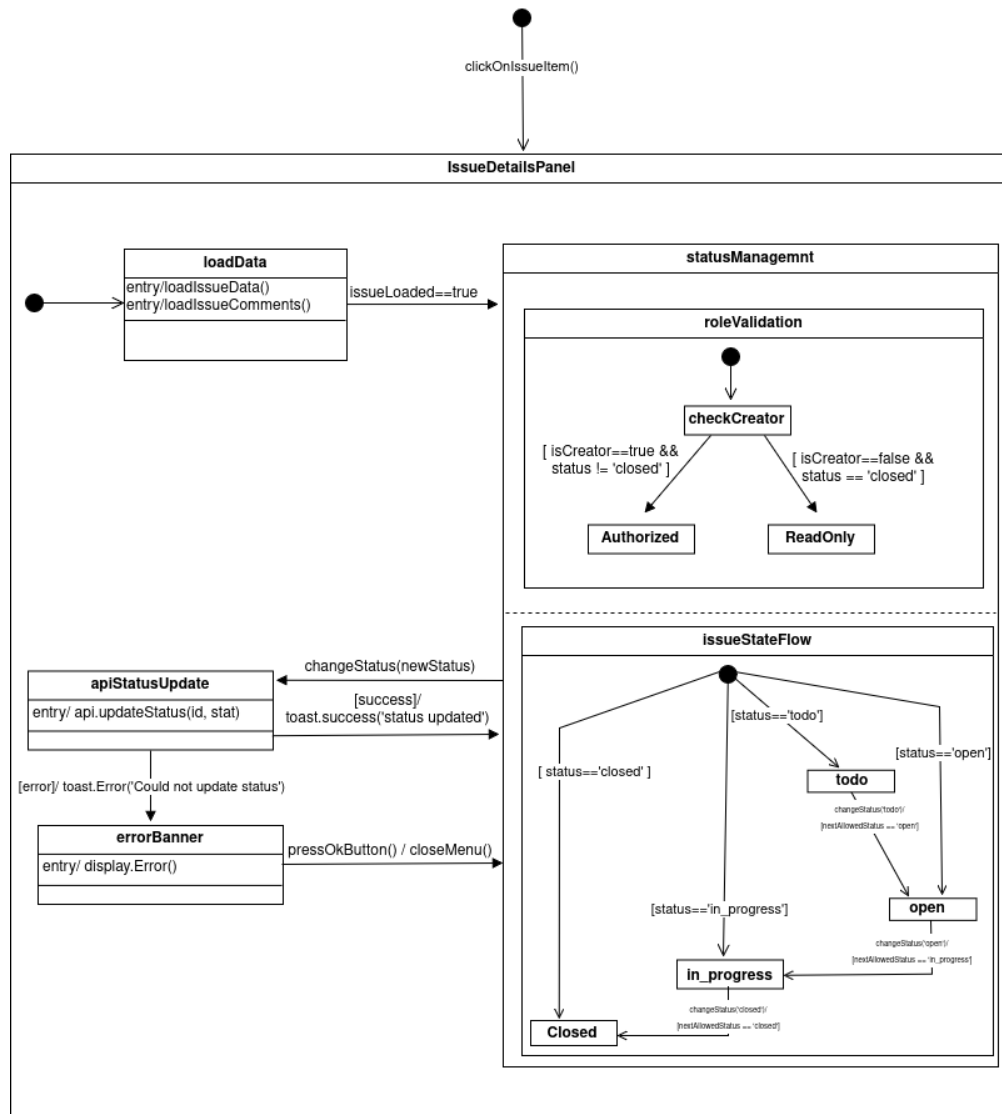


Figura 8.5: Diagramma degli stati per l'aggiornamento dello stato di una issue

Capitolo 9

Scelte di design dell'interfaccia utente

In questa sezione verranno indicate le diverse scelte di design per le interfacce.

9.1 Font e colori

Per ogni interfaccia è stato adottato un carattere di tipo **sans-serif** che risulta chiaro, semplice e ben leggibile.

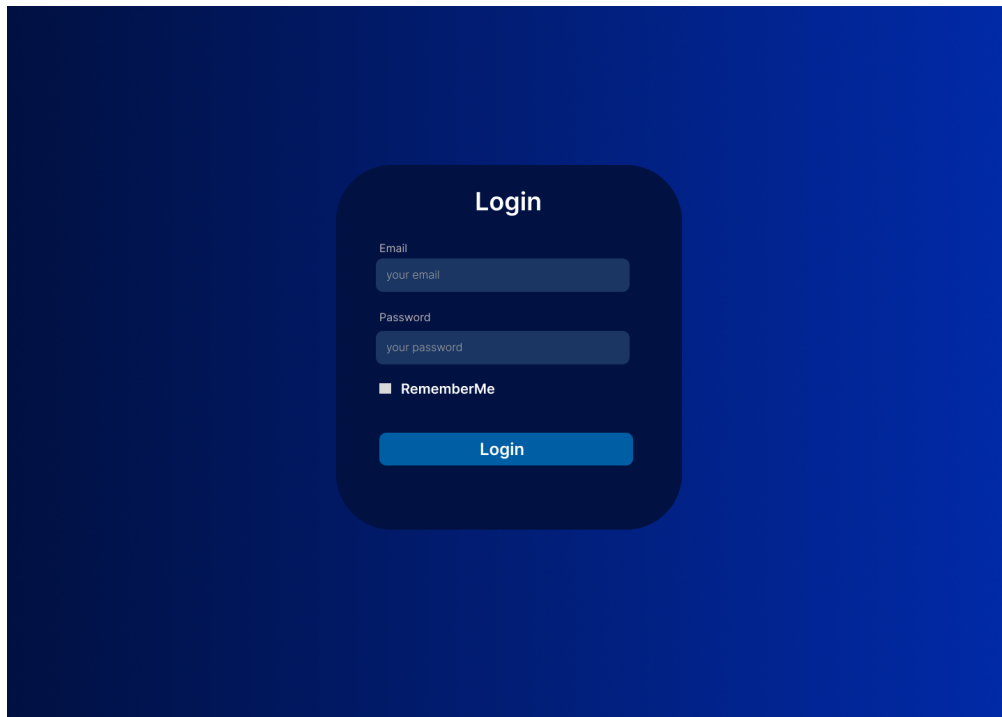


Figura 9.1: Interfaccia di login

Il sistema cromatico si basa su un approccio *Dark-mode* prevalentemente incentrato sulle sfumature del blu. Questa scelta riduce l'affaticamento visivo dell'utente.

In più i pulsanti associati ad azioni primarie e positive come **NewIssue**, **New User**, **New-Project** utilizzano una tonalità di blu più chiara e luminosa, che si stacca nettamente dallo sfondo scuro attirando l'attenzione dell'utente.

Al contrario, le azioni secondarie e distruttive come ad esempio **Cancel** hanno un contrasto attenuato per non competere visivamente con l'azione principale

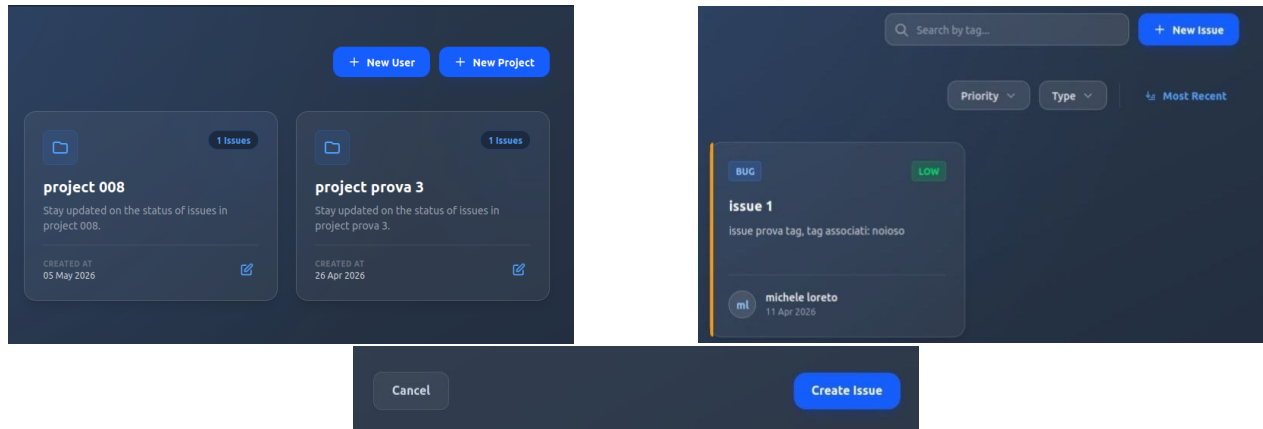


Figura 9.2: Visualizzazione bottoni

9.2 Principi della Gestalt

L'organizzazione degli elementi grafici all'interno delle schermate segue le leggi della **psicologia della Gestalt**, permettendo all'utente di comprendere le relazioni tra i dati in modo intuitivo.

9.2.1 Principio di Prossimità

Nella schermata principale dei progetti e delle issue, ogni progetto/ticket è racchiuso all'interno di una Card con bordi arrotondati e sfondo più chiaro così da permettere all'utente di percepire gli elementi all'interno della Card come un blocco logico correlato e separato dagli altri ticket/progetti adiacenti.

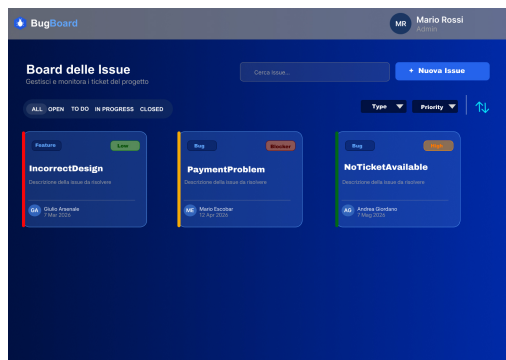


Figura 9.3: Interfaccia lista issue

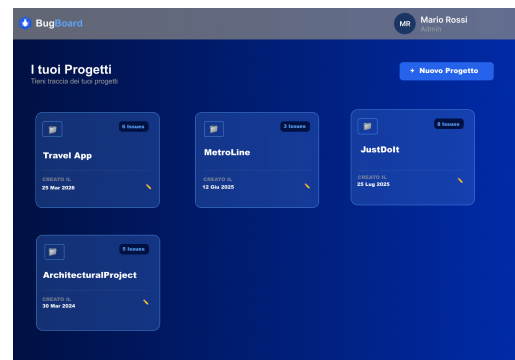


Figura 9.4: Interfaccia lista progetti

9.2.2 Euristiche di Nielsen

L'interfaccia segue anche alcune delle leggi di Nielsen per l'usabilità

9.2.3 Feedback continuo

L'applicazione garantisce che l'utente sia sempre informato sugli esiti delle proprie azioni tramite feedback visivi ma non intrusivi. Il tutto avviene principalmente tramite Toastr Notifications.

9.2.4 Consistenza

Per rispettare il principio della consistenza è stato definito un pattern per la posizione dei pulsanti a piè pagina a qualsiasi form o finestra dell'applicazione.

- Il pulsante di annullamento **Cancel** è posizionato **sempre sul lato sinistro**
- Il pulsante di conferma o salvataggio è posizionato **sempre sul lato destro**

L'adozione di questa convenzione standardizza l'esperienza d'uso: l'utente, una volta appreso il meccanismo nella prima schermata, non deve più riflettere sulla posizione dei comandi nelle pagine successive, automatizzando l'interazione e riducendo a zero il carico decisionale.

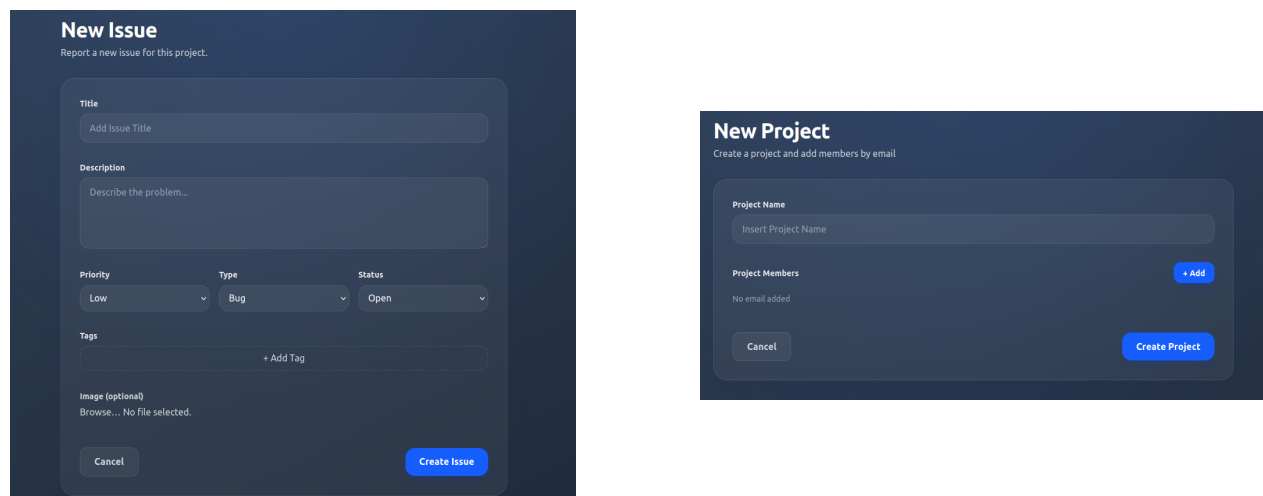


Figura 9.5: Disposizione bottoni nelle interfacce di creazione

Capitolo 10

Strumenti di versioning

Con lo scopo di garantire un efficace controllo e facilitare la collaborazione del team, il codice nel progetto è stato gestito attraverso **GitHub**

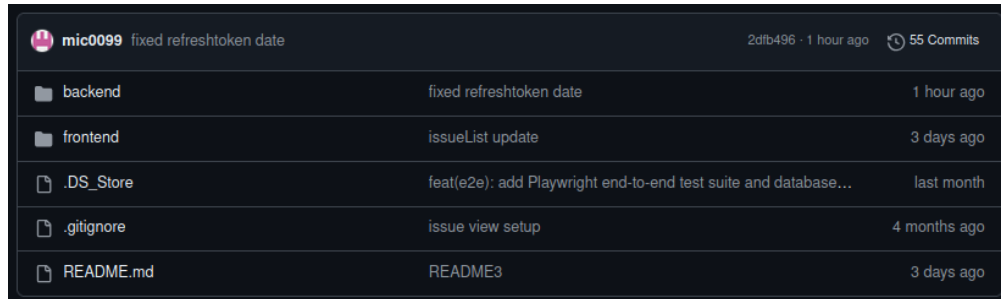


Figura 10.1: Interfaccia di github

La repository è disponibile al seguente link: <https://github.com/mic0099/BugBoard>

Commit	55
Contributors	2

Capitolo 11

SonarQube - Analisi della qualità del codice

Per assicurare *Manutenibilità*, *Affidabilità* e *Sicurezza* dell'applicazione, il codice sorgente è stato sottoposto a processi di analisi rigorosi. A tale scopo è stato impiegato **SonarQube** che ha permesso di monitorare costantemente metriche chiave quali la duplicazione del codice e la conformità agli standard di programmazione.

Di seguito vengono mostrati i **risultati finali dell'analisi**.

11.0.1 Report Finale

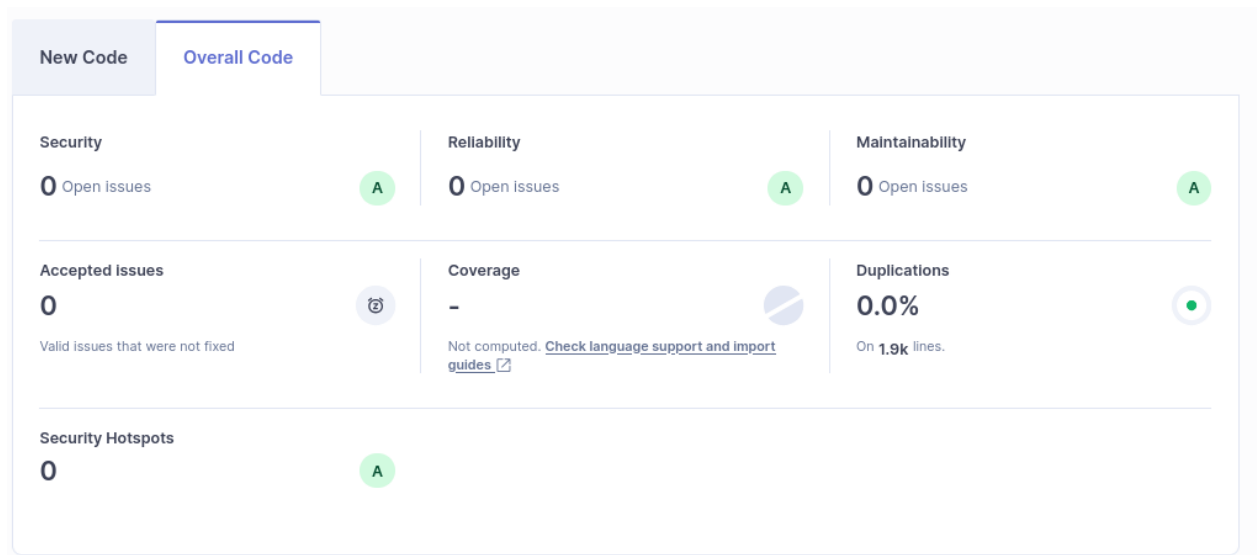


Figura 11.1: SonarQube-Report finale

Nota: All'interno della configurazione del Quality Gate di SonarQube è stata **esplicitamente esclusa la metrica relativa alla Code Coverage**. La suite di test è stata circonscritta a piccole aree del codice; di conseguenza, il calcolo della copertura globale sul resto del

codice avrebbe causato un fallimento automatico (e ingiustificato) del Quality Gate, motivo per cui si è reso necessario escluderlo dall'analisi.

11.0.2 Grafico delle issues

L'adozione di SonarQube ci ha permesso, inoltre, di individuare, tenere traccia e correggere criticità del codice durante il percorso di sviluppo, contribuendo ad un software più robusto e manutenibile.

Di seguito viene riportato il grafico delle issue rilevate:



Figura 11.2: SonarQube-Andamento temporale delle issue

Capitolo 12

Azure - Deploy dell'applicazione

L'obiettivo di questa fase del progetto è stato rendere BugBoard accessibile pubblicamente in rete, superando l'ambiente di sviluppo locale basato su Docker Compose.

12.0.1 Architettura scelta

Componente Applicativo	Servizio Azure utilizzato
Backend Express (containerizzato)	Azure Container Apps.
Database PostgreSQL	Azure Database for PostgreSQL – Flexible Server
Immagine Docker del backend	Azure Container Registry.
Frontend Angular (build statica)	Azure Storage – Static Website Hosting

Tabella 12.1: Tabella mapping tra componente app e Servizio cloud usato

Azure Container Apps è stato scelto per il backend perché è un servizio serverless container-native pensato specificamente per applicazioni containerizzate singole come la nostra.

Azure Database for PostgreSQL Flexible Server è stato scelto al posto di containerizzare Postgres perché in un ambiente di produzione il database deve essere persistente indipendentemente dal ciclo di vita del backend: un container di Container Apps non garantisce un filesystem stabile tra riavvii o redeploy.

Azure Container Registry è il registry privato dove viene caricata l'immagine Docker del backend prima che Container Apps possa eseguirla, sostituendo la build locale usata in sviluppo.

Per il frontend Angular, la scelta iniziale era **Azure Static Web Apps**, il servizio pensato esplicitamente per applicazioni frontend statiche con integrazione da GitHub. Tuttavia, in fase di creazione della risorsa, è emerso un vincolo specifico della subscription Azure for Students. Per questo motivo si è optato per un'alternativa funzionalmente equivalente per un'app puramente statica: la funzione **Static Website Hosting**, che fornisce un endpoint HTTPS pubblico su cui sono stati caricati i file generati dalla build di produzione di Angular.

Parte III

Attività di Testing

Capitolo 13

Test del sistema

13.1 Test di Unità

Al fine di garantire standard elevati di qualità e affidabilità, l'applicazione BugBoard26 è stata sottoposta a un rigoroso protocollo di validazione. L'attività di testing è stata progettata per validare l'aderenza del software ai requisiti iniziali e per garantire il corretto funzionamento di ogni componente implementato.

L'obiettivo principale è stato quello di rilevare tempestivamente eventuali anomalie o discrepanze rispetto al comportamento atteso e verificare la stabilità e la resilienza del sistema in condizioni operative simulate.

Sono stati eseguiti due tipi di approcci:

- **Analisi Black-box:** volta a validare le specifiche funzionali dall'esterno, verificando l'output del sistema a fronte di input predefiniti, indipendentemente dalla struttura del codice sottostante.
- **Analisi White-box:** focalizzata sull'ispezione della logica interna e del flusso di esecuzione, per testare la correttezza dei percorsi algoritmici e la gestione dei rami decisionali.

13.1.1 Testing Black Box del metodo `creaUser()`

Descrizione del metodo

Il metodo `creaUser` implementa il caso d'uso "Creare nuove utenze", che consente a un amministratore di registrare un nuovo profilo utente fornendo nome, cognome, email, password e livello di autorizzazione (Amministratore/Utente Standard). La suite di test verifica che il controller validi correttamente la presenza di ciascun campo obbligatorio, restituendo un errore con codice 400 in caso di parametro mancante, e che proceda alla creazione dell'utente (status 201) solo quando il payload è completo. Vengono inoltre verificati due casi limite: l'assegnazione del ruolo "Utente Standard" (admin: false, valore booleano falso ma valido) e il comportamento in presenza di più campi mancanti contemporaneamente.

Motivazione del test

Il metodo `creaUser` è stato testato perché rappresenta l'endpoint tramite cui un amministratore crea nuovi utenti, un'operazione critica per il sistema in quanto determina chi può

accedere alla piattaforma e con quali privilegi: un errore di validazione non intercettato potrebbe permettere la creazione di utenze incomplete o con permessi errati. L'approccio scelto è **black-box** perché il comportamento atteso del metodo è interamente derivabile dai requisiti, senza necessità di analizzare l'implementazione: dato un payload di input, l'esito (successo o errore) è completamente osservabile dalla risposta HTTP restituita. Seguono le tabelle che identificano le classi di equivalenza e la lista dei Test Case.

Valido/Non Valido	Classe	Esempio
Valido	Tutti i campi obbligatori presenti e corretti	Payload completo
Non Valido	Email mancante	Campo assente nel payload
Non Valido	Name mancante	Campo assente nel payload
Non Valido	Surname mancante	Campo assente nel payload
Non Valido	Password mancante	Campo assente nel payload
Non Valido	Admin mancante	Campo assente nel payload
Valido	L'utente è un amministratore	admin = true
Valido	L'utente non è un amministratore	admin = false

Tabella 13.1: Specifica dei casi di test logici per la creazione utente

Input validi

- **TC1:** richiedente amministratore, payload completo, ruolo assegnato = Amministratore → nuovo utente creato, risposta 201
- **TC2:** richiedente amministratore, payload completo, ruolo assegnato = Utente Standard → nuovo utente creato, risposta 201

Input non validi

- **TC3:** email mancante → richiesta rifiutata, errore 400
- **TC4:** name mancante → richiesta rifiutata, errore 400
- **TC5:** surname mancante → richiesta rifiutata, errore 400
- **TC6:** password mancante → richiesta rifiutata, errore 400
- **TC7:** admin mancante → richiesta rifiutata, errore 400

Combinazione di condizioni

- **TC8:** più campi obbligatori mancanti contemporaneamente (es. email e name) → la richiesta deve comunque essere rifiutata con errore 400

Eccezione

- **TC9:** creazione utente con email già esistente nel sistema -> richiesta rifiutata

```
1 // TC1: tutti i parametri validi con admin=true(successo)
2 test("should create user successfully", async () => {
3   const req = {
4     body: {
5       name: "test",
6       surname: "test",
7       email: "test@test.com",
8       password: "Test0099*",
9       admin: true
10    }
11  };
12  const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
13  const next = jest.fn();
14
15  User.create.mockResolvedValue({
16    id: 1,
17    name: "test",
18    surname: "test",
19    email: "test@test.com",
20    admin: true
21  });
22
```

```

23     await authController.creaUser(req, res, next);
24
25     expect(User.create).toHaveBeenCalledWith({
26       email: "test@test.com",
27       name: "test",
28       surname: "test",
29       password: "Test0099*",
30       admin: true
31     });
32     expect(res.status).toHaveBeenCalledWith(201);
33     expect(res.json).toHaveBeenCalledWith({
34       id: 1,
35       name: "test",
36       surname: "test",
37       email: "test@test.com",
38       admin: true
39     });
40     expect(next).not.toHaveBeenCalled();
41   });
42
43
44
45   // TC2: Tutti i parametri validi con admin = false -> successo
46   test("should create user successfully when admin is false", async () => {
47     const req = {
48       body: {
49         name: "test",
50         surname: "test",
51         email: "test@test.com",
52         password: "Test0099*",
53         admin: false
54       }
55     };
56     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
57     const next = jest.fn();
58
59     User.create.mockResolvedValue({
60       id: 2,
61       name: "test",
62       surname: "test",
63       email: "test@test.com",
64       admin: false
65     });
66
67     await authController.creaUser(req, res, next);
68
69     expect(User.create).toHaveBeenCalledWith({
70       email: "test@test.com",

```



```

71     name: "test",
72     surname: "test",
73     password: "Test0099*",
74     admin: false
75 });
76 expect(res.status).toHaveBeenCalledWith(201);
77 expect(next).not.toHaveBeenCalled();
78 });
79
80 // TC3: email mancante
81 test("should call next with error 400 if email is missing", async () => {
82     const req = {
83         body: {
84             name: "test",
85             surname: "test",
86             password: "Test0099*",
87             admin: true
88         }
89     };
90     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
91     const next = jest.fn();
92
93     await authController.creaUser(req, res, next);
94
95     expect(User.create).not.toHaveBeenCalled();
96     expect(next).toHaveBeenCalled();
97     expect(next.mock.calls[0][0].message).toBe("missing parameter email is
98         required");
99     expect(next.mock.calls[0][0].status).toBe(400);
100 });
101
102 // TC4: name mancante
103 test("should call next with error 400 if name is missing", async () => {
104     const req = {
105         body: {
106             email: "test@test.com",
107             surname: "test",
108             password: "Test0099*",
109             admin: true
110         }
111     };
112     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
113     const next = jest.fn();
114
115     await authController.creaUser(req, res, next);
116
117     expect(User.create).not.toHaveBeenCalled();
118     expect(next).toHaveBeenCalled();

```

```

118     expect(next.mock.calls[0][0].message).toBe("missing parameter name is required");
119     expect(next.mock.calls[0][0].status).toBe(400);
120 });
121
122 // TC5: surname mancante
123 test("should call next with error 400 if surname is missing", async () => {
124     const req = {
125         body: {
126             email: "test@test.com",
127             name: "test",
128             password: "Test0099*",
129             admin: true
130         }
131     };
132     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
133     const next = jest.fn();
134
135     await authController.creaUser(req, res, next);
136
137     expect(User.create).not.toHaveBeenCalled();
138     expect(next).toHaveBeenCalled();
139     expect(next.mock.calls[0][0].message).toBe("missing parameter surname is
        required");
140     expect(next.mock.calls[0][0].status).toBe(400);
141 });
142
143 // TC6: password mancante
144 test("should call next with error 400 if password is missing", async () => {
145     const req = {
146         body: {
147             name: "test",
148             surname: "test",
149             email: "test@test.com",
150             admin: true
151         }
152     };
153     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
154     const next = jest.fn();
155
156     await authController.creaUser(req, res, next);
157
158     expect(User.create).not.toHaveBeenCalled();
159     expect(next).toHaveBeenCalled();
160     expect(next.mock.calls[0][0].message).toBe("missing parameter password is
        required");
161     expect(next.mock.calls[0][0].status).toBe(400);
162 });
163

```

```

164 // TC7: admin mancante
165 test("should call next with error 400 if admin is missing", async () => {
166     const req = {
167         body: {
168             name: "test",
169             surname: "test",
170             email: "test@test.com",
171             password: "Test0099*"
172         }
173     };
174     const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
175     const next = jest.fn();
176
177     await authController.creaUser(req, res, next);
178
179     expect(User.create).not.toHaveBeenCalled();
180     expect(next).toHaveBeenCalled();
181     expect(next.mock.calls[0][0].message).toBe("missing parameter admin is
182         required");
183     expect(next.mock.calls[0][0].status).toBe(400);
184 });
185
186
187 // TC8: più parametri mancanti
188 test("should stop at first missing parameter (email) when multiple are missing",
189     async () => {
190         const req = {
191             body: {
192                 surname: "test",
193                 password: "Test0099*",
194                 admin: true
195             } // email e name mancanti
196         };
197         const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
198         const next = jest.fn();
199
200         await authController.creaUser(req, res, next);
201
202         expect(User.create).not.toHaveBeenCalled();
203         expect(next).toHaveBeenCalled();
204         expect(next.mock.calls[0][0].message).toBe("missing parameter email is
205             required");
206         expect(next.mock.calls[0][0].status).toBe(400);
207     });
208
209 // TC9: email già esistente -> User.create rigetta la Promise

```

```
209 test("should call next with error when email already exists", async () => {
210   const req = {
211     body: {
212       name: "test",
213       surname: "test",
214       email: "test@test.com",
215       password: "Test0099*",
216       admin: true
217     }
218   };
219   const res = { json: jest.fn(), status: jest.fn().mockReturnThis() };
220   const next = jest.fn();
221
222   const duplicateError = new Error("email must be unique");
223   User.create.mockRejectedValue(duplicateError);
224
225   await authController.creaUser(req, res, next);
226
227   expect(next).toHaveBeenCalledWith(duplicateError);
228   expect(res.status).not.toHaveBeenCalled();
229   expect(res.json).not.toHaveBeenCalled();
230 });
```

13.1.2 Testing black-box del metodo findIssueByTag()

Descrizione del metodo

Il metodo findIssueByTag implementa il caso d'uso "Cercare issue tramite tag", che consente a un utente autenticato di individuare le segnalazioni di un progetto tramite l'etichetta assegnata in fase di creazione. La suite verifica la validazione dei parametri di query obbligatori (content, projectId), la corretta gestione del caso in cui il tag non esista nel sistema (404), il caso limite di un tag privo di issue associate, e il comportamento di successo con la corretta interrogazione del database.

Motivazione del test

Il metodo findIssueByTag è stato testato perché implementa la funzionalità di ricerca delle segnalazioni, un'operazione centrale per l'usabilità del sistema: un errore nella gestione dei parametri di ricerca o nel filtraggio per progetto potrebbe restituire risultati errati o esporre issue di progetti non pertinenti. L'approccio scelto è **Black-box** perché il comportamento atteso — quali parametri sono richiesti, quali codici di errore restituire, cosa viene mostrato in risposta — è interamente definito dal contratto funzionale dell'endpoint e verificabile osservando solo input e output, senza necessità di analizzare la logica interna di costruzione della query.

Seguono le tabelle che identificano le classi di equivalenza e la lista dei Test Case.

Valido/Non Valido	Classe	Esempio
Valido	tag presente e corrispondente a issue esistenti	content: "bug"
Non Valido	tag mancante	Parametro assente
Non Valido	identificativo del progetto mancante	Parametro assente
Non Valido	tag non esistente nel sistema	Nessuna corrispondenza trovata
Valido	identificativo del progetto presente	projectId: 3
Valido	tag esistente ma nessuna issue associata al progetto specificato	Risultato vuoto

Tabella 13.2: Specifica dei casi di test logici per la ricerca di una issue

Input validi

- **TC1:** tag e progetto validi, issue corrispondenti presenti → risposta 200 con elenco delle issue trovate
- **TC2(limite):** tag e progetto validi, nessuna issue associata → risposta 200 con elenco vuoto

Input non validi

- **TC3:** tag mancante → richiesta rifiutata, errore 400
- **TC4:** identificativo progetto mancante → richiesta rifiutata, errore 400
- **TC5:** tag inesistente nel sistema → richiesta rifiutata, errore 404

Combinazione di condizioni

- **TC6:** tag e identificativo progetto entrambi mancanti → richiesta comunque rifiutata con errore 400

Gestione errori non previsti

- **TC4:** req.user assente/malformato → TypeError nativo, catturato comunque dal blocco catch generico e propagato correttamente a next — copre un path aggiuntivo non derivante da decisioni esplicite nel codice, ma dalla presenza del blocco try/catch

```

1 // TC1: content + projectId validi, tag e issue trovati
2 test("findIssueByTag returns issues correctly", async () => {
3   const req = {
4     query: {
5       content: "bug",
6       projectId: 3
7     }
8   };
9
10  const res = {
11    json: jest.fn(),
12    status: jest.fn().mockReturnThis()
13  };
14
15  const next = jest.fn();
16
17  const Issues = [
18    { title: "issue test" }
19  ];
20
21  mockGetIssues.mockResolvedValue(Issues);
22
23  Tag.findOne.mockResolvedValue({
24    getIssues: mockGetIssues

```

```

25 });
26
27 await issueController.findIssueByTag(req, res, next);
28
29 expect(Tag.findOne).toHaveBeenCalledWith({
30   where: { content: "bug" }
31 });
32
33 expect(mockGetIssues).toHaveBeenCalledWith({
34   where: { projectId: 3 },
35   include: [
36     { model: expect.anything(), attributes: ['name', 'surname'] },
37     { model: expect.anything(), attributes: ['name'] },
38     { model: expect.anything() }
39   ],
40   joinTableAttributes: []
41 });
42
43 expect(res.status).toHaveBeenCalledWith(200);
44 expect(res.json).toHaveBeenCalledWith(Issues);
45 expect(next).not.toHaveBeenCalled();
46 });
47
48
49
50
51 // TC2: tag trovato ma nessuna issue collegata (array vuoto)
52 test("returns 200 with empty array when tag has no issues", async () => {
53   const req = {
54     query: {
55       content: "bug",
56       projectId: 3
57     }
58   };
59
60   const res = {
61     json: jest.fn(),
62     status: jest.fn().mockReturnThis()
63   };
64
65   const next = jest.fn();
66
67   mockGetIssues.mockResolvedValue([]);
68   Tag.findOne.mockResolvedValue({ getIssues: mockGetIssues });
69
70   await issueController.findIssueByTag(req, res, next);
71
72   expect(res.status).toHaveBeenCalledWith(200);

```

```

73     expect(res.json).toHaveBeenCalled();
74     expect(next).not.toHaveBeenCalled();
75 });
76
77
78 // TC3: content mancante
79 test("calls next with error when content (tag) is missing", async () => {
80     const req = {
81         query: {
82             projectId: 3
83         }
84     };
85
86     const res = {
87         json: jest.fn(),
88         status: jest.fn().mockReturnThis()
89     };
90
91     const next = jest.fn();
92
93     await issueController.findIssueByTag(req, res, next);
94
95     expect(Tag.findOne).not.toHaveBeenCalled();
96     expect(mockGetIssues).not.toHaveBeenCalled();
97     expect(res.status).not.toHaveBeenCalled();
98     expect(res.json).not.toHaveBeenCalled();
99
100    expect(next).toHaveBeenCalled();
101    expect(next.mock.calls[0][0].message).toBe("missing tag");
102    expect(next.mock.calls[0][0].status).toBe(400);
103 });
104
105 // TC4: projectId mancante
106 test("calls next with error when projectId is missing", async () => {
107     const req = {
108         query: {
109             content: "bug"
110         }
111     };
112
113     const res = {
114         json: jest.fn(),
115         status: jest.fn().mockReturnThis() // corretto
116     };
117
118     const next = jest.fn();
119
120     await issueController.findIssueByTag(req, res, next);

```



```

121
122 expect(Tag.findOne).not.toHaveBeenCalled();
123 expect(mockGetIssues).not.toHaveBeenCalled();
124 expect(res.status).not.toHaveBeenCalled();
125 expect(res.json).not.toHaveBeenCalled();
126
127 expect(next).toHaveBeenCalled();
128 expect(next.mock.calls[0][0].message).toBe("missing projectId");
129 expect(next.mock.calls[0][0].status).toBe(400);
130 });
131
132
133
134 // TC5: projectId non numerico
135 test("calls next with error 400 when projectId is not a number", async () => {
136   const req = {
137     query: {
138       content: "bug",
139       projectId: "abc"
140     }
141   };
142
143   const res = {
144     json: jest.fn(),
145     status: jest.fn().mockReturnThis()
146   };
147
148   const next = jest.fn();
149
150   await issueController.findIssueByTag(req, res, next);
151
152   expect(Tag.findOne).not.toHaveBeenCalled();
153   expect(mockGetIssues).not.toHaveBeenCalled();
154   expect(res.status).not.toHaveBeenCalled();
155   expect(res.json).not.toHaveBeenCalled();
156
157   expect(next).toHaveBeenCalled();
158   expect(next.mock.calls[0][0].message).toBe("projectId must be a number");
159   expect(next.mock.calls[0][0].status).toBe(400);
160 });
161
162
163
164
165 // TC6: tag non trovato
166 test("calls next with error 404 when tag is not found", async () => {
167   const req = {
168     query: {

```

```

169         content: "bug",
170         projectId: 3
171     }
172 };
173
174 const res = {
175     status: jest.fn().mockReturnThis(),
176     json: jest.fn()
177 };
178
179 const next = jest.fn();
180
181 Tag.findOne.mockResolvedValue(null);
182
183 await issueController.findIssueByTag(req, res, next);
184
185 expect(Tag.findOne).toHaveBeenCalledWith({
186     where: { content: "bug" }
187 });
188
189 expect(mockGetIssues).not.toHaveBeenCalled();
190 expect(res.status).not.toHaveBeenCalled();
191 expect(res.json).not.toHaveBeenCalled();
192
193 expect(next).toHaveBeenCalled();
194 expect(next.mock.calls[0][0].message).toBe("tag not found");
195 expect(next.mock.calls[0][0].status).toBe(404);
196 });

```

13.1.3 Testing white-box del metodo getAllProjects()

Descrizione del metodo

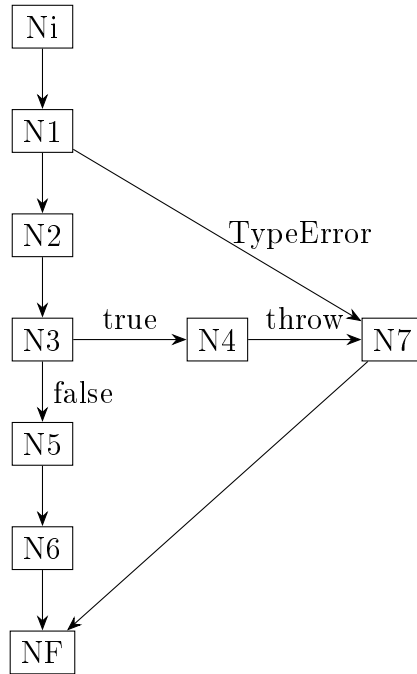
Il metodo `getAllProjects` implementa il caso d'uso "Visualizzare lista riepilogativa progetti", restituendo all'utente autenticato l'elenco dei progetti a cui è assegnato, arricchito con il conteggio delle issue associate a ciascun progetto. La suite verifica sia il comportamento esterno (gestione utente inesistente, caso limite di assenza progetti, risposta di successo), sia la correttezza strutturale della query Sequelize costruita internamente (clausole `group`, `order`, `include`, `attributes`, `subQuery`), necessaria per l'aggregazione corretta dei dati.

Motivazione del test

Il metodo `getAllProjects` è stato testato perché fornisce la dashboard principale dei progetti dell'utente, un dato su cui si basa la navigazione dell'intera applicazione: un errore nella query di aggregazione produrrebbe conteggi delle issue errati o un ordinamento non conforme al requisito, senza generare alcun errore visibile a runtime. L'approccio scelto è **white-box** perché la parte più critica del metodo — la costruzione della query di aggregazione — non è verificabile osservando solo l'output di un test unitario basato su mock: un mock restituisce sempre il valore predefinito indipendentemente dai parametri ricevuti, quindi un difetto nella

query (es. un campo mancante nel group o un ordinamento invertito) non produrrebbe alcun fallimento in un test che osservasse solo lo stato e il corpo della risposta. Solo l'ispezione diretta dei parametri passati alla funzione mockata consente di rilevare questa classe di difetti.

Control Flow Graph



Cammini Indipendenti

Path	Percorso	Descrizione
P1	N1 -> N2 -> N3(false) ->N5 -> N6	Utente trovato, flusso completato con successo
P2	N1 -> N2 -> N3 (true) -> N4 -> N7	Utente non trovato, errore gestito esplicitamente

Tabella 13.3: Percorsi Indipendenti

Classi di Equivalenza

Valido/Non Valido	Classe	Esempio
Valido	Utente esistente	risultato di <code>User.findByPk</code> non nullo
Non Valido	Utente inesistente	<code>User.findByPk</code> restituisce null
Valido	Utente con progetti associati	array non vuoto restituito da <code>getProjects</code>
Valido	Utente senza progetti associati	array vuoto restituito da <code>getProjects</code>
-	Struttura della query costruita dal metodo	attributes, include, group, order, subQuery passati a <code>getProjects</code>

Tabella 13.4: Specifica dei casi di test logici per i progetti di un utente

Nota: L'ultima riga non è una classe di equivalenza in senso stretto sui dati di input, ma rappresenta l'oggetto di verifica interna che giustifica l'approccio white-box: non varia in funzione dell'input dell'utente (è sempre la stessa struttura), ma va comunque validata perché costituisce la parte più critica e meno "visibile" del metodo.

Input validi

- **TC1:** utente esistente, con progetti associati → status 200, risposta con l'array di progetti; inoltre, verifica puntuale della struttura della query passata a `getProjects` (group, order, subQuery, attributes, include), per garantire la correttezza interna della logica di aggregazione — copre il path P1

Input non validi

- **TC2:** utente inesistente (`User.findByPk` restituisce null) → `next(err)`, messaggio "User not found", status 404; verifica che `getProjects` non venga mai invocato — copre il path P2

Casi limite del successo

- **TC3:** utente esistente, ma senza progetti associati → status 200, risposta con array vuoto — variante di P1, verifica che l'assenza di risultati non comprometta il flusso previsto

Eccezione

- **TC4:** req.user assente/malformato → TypeError nativo, catturato comunque dal blocco catch generico e propagato correttamente a next.

```
1 // TC1: utente trovato, con progetti (successo)
2 test("should return all projects of user", async () => {
3   const mockProjects = [
4     { projectId: 1, name: "BugBoard" }
5   ];
6
7   const mockUser = {
8     getProjects: jest.fn().mockResolvedValue(mockProjects)
9   };
10
11   User.findByPk.mockResolvedValue(mockUser);
12
13   const req = {
14     user: { userId: 1 }
15   };
16
17   const res = {
18     json: jest.fn(),
19     status: jest.fn().mockReturnThis()
20   };
21
22   const next = jest.fn();
23
24   await projectController.getAllProjects(req, res, next);
25
26   expect(User.findByPk).toHaveBeenCalledWith(1);
27   expect(mockUser.getProjects).toHaveBeenCalledTimes(1);
28
29   const callArgs = mockUser.getProjects.mock.calls[0][0];
30
31   expect(callArgs.group).toEqual([
32     'Project.projectId',
33     'Project.name',
34     'Project.createdAt',
35     'Users.userId',
36     'Users.email',
37     'Users.name',
38     'Users.surname',
39     'UserProjects.userId',
40     'UserProjects.projectId',
41     'UserProjects.createdAt',
42     'UserProjects.updatedAt'
43   ]);
44
45   expect(callArgs.order).toEqual(['name', 'ASC']);
```

```

46 expect(callArgs.subQuery).toBe(false);
47
48
49 expect(callArgs.include).toHaveLength(2);
50 expect(callArgs.include[0]).toMatchObject({
51   attributes: [],
52   required: false
53 });
54 expect(callArgs.include[1]).toMatchObject({
55   attributes: ['email', 'name', 'surname'],
56   through: { attributes: [] },
57   required: false
58 });
59
60 expect(callArgs.attributes[0]).toBe('projectId');
61 expect(callArgs.attributes[1]).toBe('name');
62 expect(callArgs.attributes[2]).toBe('createdAt');
63 expect(callArgs.attributes).toHaveLength(4);
64 expect(callArgs.attributes[3][1]).toBe('issuesCount');
65
66 expect(res.status).toHaveBeenCalledWith(200);
67 expect(res.json).toHaveBeenCalledWith(mockProjects);
68 expect(next).not.toHaveBeenCalled();
69 });
70
71 // TC2: utente non trovato
72 test("should return 404 if user does not exist", async () => {
73   User.findByPk.mockResolvedValue(null);
74
75   const req = {
76     user: { userId: 1 }
77   };
78
79   const res = {
80     json: jest.fn(),
81     status: jest.fn().mockReturnThis()
82   };
83
84   const next = jest.fn();
85
86   await projectController.getAllProjects(req, res, next);
87
88   expect(User.findByPk).toHaveBeenCalledWith(1);
89
90   expect(next).toHaveBeenCalled();
91   expect(next.mock.calls[0][0].message).toBe("User not found");
92   expect(next.mock.calls[0][0].status).toBe(404);
93

```

```

94
95     expect(res.status).not.toHaveBeenCalled();
96     expect(res.json).not.toHaveBeenCalled();
97 });
98
99 // TC3: utente trovato, ma nessun progetto associato
100 test("should return 200 with empty array when user has no projects", async () => {
101     const mockUser = {
102         getProjects: jest.fn().mockResolvedValue([])
103     };
104
105     User.findByPk.mockResolvedValue(mockUser);
106
107     const req = {
108         user: { userId: 1 }
109     };
110
111     const res = {
112         json: jest.fn(),
113         status: jest.fn().mockReturnThis()
114     };
115
116     const next = jest.fn();
117
118     await projectController.getAllProjects(req, res, next);
119
120     expect(mockUser.getProjects).toHaveBeenCalledTimes(1);
121     expect(res.status).toHaveBeenCalledWith(200);
122     expect(res.json).toHaveBeenCalledWith([]);
123     expect(next).not.toHaveBeenCalled();
124 });
125
126
127
128 // TC4 (Eccezione): req.user assente/malformato (TypeError catturato dal catch
    generico)
129 test("calls next with error when req.user is missing", async () => {
130     const req = {};
131
132     const res = {
133         json: jest.fn(),
134         status: jest.fn().mockReturnThis()
135     };
136
137     const next = jest.fn();
138
139     await projectController.getAllProjects(req, res, next);
140

```

```
141     expect(User.findByPk).not.toHaveBeenCalled();
142     expect(res.status).not.toHaveBeenCalled();
143     expect(res.json).not.toHaveBeenCalled();
144
145     expect(next).toHaveBeenCalled();
146     expect(next.mock.calls[0][0]).toBeInstanceOf(TypeError);
147   });
```

13.2 Test End-to-End

Sebbene i test End-to-End (E2E) non fossero esplicitamente richiesti dalle specifiche di progetto, si è ritenuto opportuno svilupparli in considerazione della natura web dell'applicazione realizzata.

I test unitari (JUnit) consentono di verificare la correttezza delle singole componenti software, ma non risultano sufficienti a validare scenari di integrazione che coinvolgono l'interfaccia utente, la navigazione tra le pagine e le comunicazioni HTTP asincrone.

Tali aspetti rappresentano i flussi operativi più significativi dal punto di vista dell'esperienza dell'utente e richiedono pertanto una validazione a livello applicativo.

Per l'implementazione dei test E2E è stato adottato il framework Playwright.

13.2.1 Setup dell'ambiente di test

Al fine di garantire l'esecuzione affidabile e riproducibile dei test End-to-End, è stata predisposta una configurazione dedicata della suite di testing. In particolare, sono stati introdotti i seguenti componenti:

- **Autenticazione automatica**

Prima dell'esecuzione della suite viene eseguito uno script di setup (`auth.setup.ts`) che effettua automaticamente il login all'applicazione utilizzando un account di test. Lo stato della sessione autenticata viene successivamente salvato da Playwright (`storageState`) e riutilizzato dai test, evitando di ripetere la procedura di autenticazione per ogni caso di test. Questo approccio riduce i tempi di esecuzione e garantisce che tutti i test partano da uno stato autenticato consistente.

- **Generazione di dati univoci**

Per evitare conflitti tra diverse esecuzioni della suite, i dati creati durante i test (ad esempio progetti e issue) vengono identificati mediante nomi univoci ottenuti concatenando un timestamp al nome dell'entità. In questo modo ogni esecuzione risulta indipendente dalle precedenti.

- **Pulizia automatica del database**

Al termine dell'esecuzione della suite viene eseguito uno script di pulizia (`cleanupE2E.js`) che elimina automaticamente tutti i dati generati dai test. Lo script rimuove le entità create durante l'esecuzione (progetti, issue, commenti, immagini, tag e relative associazioni), ripristinando il database nello stato iniziale.

13.2.2 Test di Autenticazione Utente

Obiettivo

L'obiettivo del test è verificare il corretto funzionamento del processo di autenticazione dell'applicazione BugBoard. In particolare, il test controlla che un utente registrato possa effettuare il login utilizzando credenziali valide e accedere correttamente all'area riservata dell'applicazione.

La verifica interessa l'intero flusso di autenticazione, comprendendo sia l'interazione dell'utente con l'interfaccia grafica sia la corretta gestione delle credenziali da parte del sistema. Il test accerta che, a seguito dell'inserimento di dati validi, l'applicazione consenta l'accesso all'utente autenticato e lo reindirizzi alla dashboard principale.

Per garantire l'indipendenza del test da eventuali sessioni precedenti, l'esecuzione viene effettuata utilizzando uno stato del browser privo di cookie e dati di autenticazione persistenti. In questo modo il test verifica esplicitamente il processo di login senza beneficiare di sessioni già attive.

Flusso di Esecuzione

Il test viene eseguito secondo la seguente sequenza operativa:

1. Il browser viene inizializzato senza cookie o informazioni di sessione precedentemente memorizzate.
2. L'utente viene reindirizzato alla pagina di autenticazione dell'applicazione (`/login`).
3. Viene verificato che la navigazione sia avvenuta correttamente controllando l'URL della pagina corrente.
4. Il campo relativo all'indirizzo e-mail viene compilato con le credenziali dell'utente di test `playwright_admin@test.it`.
5. Il campo relativo alla password viene compilato con la password associata all'account di test.
6. Viene selezionata l'opzione *Remember Me*, simulando il comportamento di un utente che desidera mantenere la sessione attiva anche dopo la chiusura del browser.
7. Il form di autenticazione viene inviato tramite il pulsante di login presente nell'interfaccia.
8. Il test attende il completamento del processo di autenticazione e il conseguente reindirizzamento dell'utente.
9. Una volta completato il login, viene verificato il corretto accesso alla dashboard dei progetti (`/projectList`).
10. Infine, viene controllata la presenza di un elemento testuale identificativo della dashboard, al fine di confermare il caricamento corretto dell'area riservata dell'applicazione.

Assertions

Il test è considerato superato esclusivamente se tutte le seguenti condizioni risultano verificate:

- L'applicazione consente l'accesso alla pagina di login.

- L'inserimento delle credenziali valide non genera errori di autenticazione.
- L'invio del form di login viene elaborato correttamente dal sistema.
- A seguito dell'autenticazione, l'utente viene automaticamente reindirizzato alla pagina `/projectList`.
- La dashboard principale risulta correttamente caricata e accessibile.
- Il testo identificativo *Project Dashboard* è visibile all'interno della pagina, confermando che l'utente ha effettivamente ottenuto accesso all'area autenticata dell'applicazione.

Nel complesso, il test realizza una verifica End-to-End del meccanismo di autenticazione, validando l'intero processo che coinvolge l'interfaccia utente, la gestione delle credenziali, la creazione della sessione autenticata e il corretto accesso alle funzionalità riservate agli utenti registrati.

13.2.3 Test di Creazione di una Nuova Issue

Obiettivo

L'obiettivo del test è verificare il corretto funzionamento della procedura di creazione di una nuova issue all'interno di un progetto dell'applicazione BugBoard.

La funzionalità di creazione delle issue costituisce uno dei principali casi d'uso del sistema, consentendo agli utenti di registrare bug, segnalazioni e attività da monitorare all'interno di uno specifico progetto. Il test verifica che un utente autenticato possa creare correttamente una nuova issue compilando il form di inserimento e che le informazioni fornite vengano correttamente elaborate dal backend, memorizzate nel database e successivamente rese disponibili nell'interfaccia utente.

Per garantire una copertura completa della funzionalità sono state implementate due varianti del test:

- **Creazione di una issue semplice**, contenente esclusivamente i campi obbligatori richiesti dal sistema.
- **Creazione di una issue con tag e immagine**, che verifica la corretta gestione delle informazioni opzionali associate alla issue.

Entrambe le varianti condividono lo stesso flusso principale di creazione della issue e differiscono esclusivamente per la presenza di metadati aggiuntivi e di un allegato multimediale.

Flusso di Esecuzione

Il test viene eseguito secondo la seguente sequenza operativa:

1. L'utente accede alla dashboard dei progetti dell'applicazione.
2. Per garantire l'indipendenza del test rispetto ai dati presenti nel database, viene creato un nuovo progetto di test con un nome generato dinamicamente.

3. Durante la creazione del progetto viene associato un membro tramite indirizzo e-mail e viene verificata la corretta risposta dell'endpoint di creazione.
4. Una volta completata la creazione del progetto, il test accede alla relativa pagina di gestione delle issue.
5. Tramite il pulsante dedicato viene aperto il form di creazione di una nuova issue.
6. Vengono compilati i campi obbligatori della issue, inclusi titolo, descrizione, priorità, tipologia e stato.
7. Prima dell'invio del form, il test registra un listener per intercettare la richiesta HTTP POST diretta all'endpoint responsabile della creazione della issue.
8. Il form viene inviato mediante il pulsante di conferma.
9. Il test attende la risposta del server e verifica l'esito dell'operazione.
10. Al termine della procedura l'applicazione reindirizza l'utente alla pagina contenente l'elenco delle issue del progetto.
11. La pagina viene ricaricata e sincronizzata con il backend per garantire che i dati persistiti siano effettivamente disponibili.
12. Vengono recuperati tutti i titoli delle issue presenti nell'interfaccia e confrontati con il titolo generato durante il test.

Per la variante avanzata del test vengono inoltre eseguite le seguenti operazioni aggiuntive:

13. Viene aggiunto un tag descrittivo alla issue mediante l'apposita funzionalità di gestione dei tag.
14. Viene selezionato e caricato un file immagine utilizzando il componente di upload previsto dall'applicazione.
15. Dopo la creazione della issue, il test accede alla relativa pagina di dettaglio.
16. Viene verificata la presenza dell'immagine associata alla issue.
17. Il test attende il completamento del caricamento dell'immagine e controlla che il file sia stato effettivamente scaricato e renderizzato dal browser.

Assertions

Il test è considerato superato esclusivamente se tutte le seguenti condizioni risultano verificate:

- La creazione del progetto preliminare viene completata con successo.
- L'accesso alla pagina di creazione della issue avviene correttamente.

- L'invio del form genera una richiesta HTTP POST verso l'endpoint `/addIssue`.
- Il server restituisce il codice di stato HTTP 201 `Created`, indicando la corretta creazione della risorsa.
- L'applicazione reindirizza correttamente l'utente alla lista delle issue associate al progetto.
- La nuova issue compare nell'elenco delle issue visualizzate dall'interfaccia.
- Il titolo della issue presente nella lista coincide con quello generato durante l'esecuzione del test.
- Nel caso della variante estesa, il tag associato viene correttamente elaborato e salvato dal sistema.
- Nel caso della variante estesa, l'immagine allegata viene correttamente caricata e associata alla issue.
- Nel caso della variante estesa, l'immagine risulta visibile nella pagina di dettaglio della issue.
- Nel caso della variante estesa, il browser conferma il completamento del caricamento dell'immagine verificando che il file sia stato renderizzato correttamente.

Nel complesso, il test realizza una verifica End-to-End della funzionalità di creazione delle issue, validando l'intero flusso applicativo che coinvolge la creazione del progetto, l'inserimento dei dati della issue, la comunicazione con il backend, la persistenza delle informazioni nel database, la gestione di tag e allegati multimediali e la successiva visualizzazione delle informazioni nell'interfaccia utente.

13.2.4 Test di Creazione di un Nuovo Progetto

Obiettivo

L'obiettivo del test è verificare il corretto funzionamento della procedura di creazione di un nuovo progetto all'interno dell'applicazione BugBoard. In particolare, il test controlla che un utente autenticato possa creare un progetto specificando un nome univoco e associando un membro al progetto tramite indirizzo e-mail.

La verifica non si limita al corretto comportamento dell'interfaccia utente, ma include anche la validazione dell'interazione tra front-end e back-end. Il test accerta infatti che la richiesta HTTP di creazione venga elaborata con successo dal server e che il nuovo progetto venga effettivamente persistito e successivamente visualizzato nella dashboard dei progetti.

Per evitare interferenze tra esecuzioni successive, il nome del progetto viene generato dinamicamente utilizzando il timestamp corrente, garantendo l'unicità dei dati di test.

Flusso di Esecuzione

Il test viene eseguito secondo la seguente sequenza operativa:

1. L'utente viene reindirizzato alla pagina contenente la lista dei progetti (`/projectList`).
2. Viene verificato che la navigazione sia avvenuta correttamente controllando l'URL della pagina corrente.
3. Tramite il pulsante dedicato alla creazione di un nuovo progetto, il test accede alla pagina di inserimento dati (`/newproject`).
4. Viene verificata la corretta apertura della pagina di creazione del progetto.
5. Il campo relativo al nome del progetto viene compilato con un valore generato dinamicamente al momento dell'esecuzione.
6. Attraverso l'apposita funzionalità dell'interfaccia, viene aggiunto un membro al progetto inserendo l'indirizzo e-mail dell'utente di test `playwright_admin@test.it`.
7. Prima dell'invio del form, il test registra un listener che rimane in attesa della richiesta HTTP `POST` verso l'endpoint responsabile della creazione del progetto (`/addProject`).
8. Il form viene inviato mediante il pulsante di conferma.
9. Il test attende la ricezione della risposta HTTP associata alla richiesta di creazione.
10. Dopo il completamento dell'operazione, viene verificato il reindirizzamento automatico alla pagina contenente la lista dei progetti.
11. Per garantire la sincronizzazione con lo stato effettivamente persistito sul server, la pagina viene ricaricata e il test attende il completamento di tutte le richieste di rete pendenti.
12. Vengono quindi recuperati tutti i nomi dei progetti presenti nella dashboard tramite gli elementi identificati dal relativo `data-testid`.
13. I valori ottenuti vengono normalizzati eliminando eventuali spazi superflui e successivamente confrontati con il nome del progetto creato durante l'esecuzione del test.

Assertions

Il test è considerato superato esclusivamente se tutte le seguenti condizioni risultano verificate:

- La navigazione iniziale verso la pagina di elenco dei progetti avviene correttamente.
- L'apertura della pagina di creazione del progetto reindirizza correttamente all'URL previsto.
- L'invio del form genera una richiesta HTTP di tipo `POST` verso l'endpoint `/addProject`.

- Il server restituisce il codice di stato HTTP 201 `Created`, indicando la corretta creazione della risorsa.
- Al termine dell'operazione l'utente viene automaticamente reindirizzato alla dashboard dei progetti.
- Dopo il ricaricamento della pagina, il progetto appena creato risulta presente nell'elenco dei progetti visualizzati dall'applicazione.
- Il nome del progetto recuperato dall'interfaccia coincide esattamente con quello generato dal test, confermando la corretta persistenza dei dati nel database e la loro successiva visualizzazione nel front-end.

13.2.5 Test di Creazione di un Nuovo Commento

Obiettivo

L'obiettivo del test è verificare il corretto funzionamento della funzionalità di inserimento dei commenti all'interno dell'applicazione BugBoard.

La possibilità di aggiungere commenti ad una issue rappresenta uno degli strumenti principali di collaborazione tra gli utenti, consentendo di fornire aggiornamenti, chiarimenti e informazioni aggiuntive relative ad uno specifico problema o attività.

Il test verifica che un utente autenticato possa creare correttamente un nuovo commento associato ad una issue esistente e che il contenuto inserito venga correttamente elaborato dal backend, memorizzato nel database e successivamente visualizzato nell'interfaccia utente.

Per garantire l'indipendenza dell'esecuzione da dati preesistenti, il test provvede autonomamente alla creazione delle entità necessarie, generando dinamicamente un nuovo progetto e una nuova issue prima di procedere con l'inserimento del commento.

Flusso di Esecuzione

Il test viene eseguito secondo la seguente sequenza operativa:

1. L'utente accede alla dashboard dei progetti dell'applicazione.
2. Per garantire l'isolamento del test, viene creato un nuovo progetto utilizzando un nome generato dinamicamente.
3. Prima dell'invio del form di creazione del progetto, viene registrato un listener per intercettare la richiesta HTTP `POST` diretta all'endpoint responsabile della creazione del progetto.
4. Il progetto viene creato e il test verifica il corretto completamento dell'operazione.
5. Una volta completata la creazione del progetto, la pagina viene ricaricata e sincronizzata con il backend.
6. Il test individua il progetto appena creato e accede alla relativa area di gestione delle issue.

7. Tramite il pulsante dedicato viene aperto il form di creazione di una nuova issue.
8. Vengono compilati tutti i campi obbligatori della issue, inclusi titolo, descrizione, priorità, tipologia e stato.
9. Prima dell'invio del form viene registrato un listener per intercettare la richiesta HTTP `POST` associata alla creazione della issue.
10. Il form viene inviato e il test attende la risposta del server.
11. Dalla risposta ricevuta viene estratto l'identificativo univoco della issue appena creata.
12. Utilizzando tale identificativo, il test accede direttamente alla pagina dedicata ai commenti della issue.
13. Viene compilato il campo di inserimento del commento con un contenuto generato dinamicamente.
14. Prima dell'invio del commento viene registrato un listener per intercettare la richiesta HTTP `POST` responsabile della creazione del commento.
15. Il commento viene inviato tramite l'apposito pulsante di conferma.
16. Il test attende la risposta del server e verifica il completamento dell'operazione.
17. La pagina viene ricaricata per sincronizzare l'interfaccia con lo stato persistito nel database.
18. Vengono recuperati tutti i commenti presenti nella pagina e confrontati con il contenuto generato durante il test.

Assertions

Il test è considerato superato esclusivamente se tutte le seguenti condizioni risultano verificate:

- La creazione del progetto preliminare viene completata con successo.
- Il server restituisce il codice di stato HTTP `201 Created` in risposta alla richiesta di creazione del progetto.
- La creazione della issue necessaria al test viene completata correttamente.
- Il server restituisce il codice di stato HTTP `201 Created` in risposta alla richiesta di creazione della issue.
- L'accesso alla pagina dei commenti della issue avviene correttamente.
- L'invio del commento genera una richiesta HTTP `POST` verso l'endpoint responsabile della creazione del commento.

- Il server restituisce il codice di stato HTTP 201 `Created`, indicando la corretta creazione della nuova risorsa.
- Dopo il ricaricamento della pagina, il commento appena inserito risulta presente nell'elenco dei commenti visualizzati dall'applicazione.
- Il contenuto del commento recuperato dall'interfaccia coincide con il valore generato durante l'esecuzione del test.

Nel complesso, il test realizza una verifica End-to-End della funzionalità di gestione dei commenti, validando l'intero flusso applicativo che coinvolge la creazione delle entità necessarie, la comunicazione tra frontend e backend, la persistenza dei dati nel database e la successiva visualizzazione del commento nell'interfaccia utente.

13.2.6 Test di Ricerca delle Issue tramite Tag

Obiettivo

L'obiettivo del test è verificare il corretto funzionamento della funzionalità di ricerca delle issue mediante tag all'interno dell'applicazione BugBoard.

I tag rappresentano uno strumento fondamentale per la classificazione e l'organizzazione delle issue, consentendo agli utenti di filtrare rapidamente le segnalazioni in base a specifiche categorie o caratteristiche. Il test verifica che il sistema sia in grado di individuare correttamente le issue associate ad un determinato tag e di mostrare all'utente esclusivamente i risultati pertinenti alla ricerca effettuata.

Per garantire l'indipendenza dell'esecuzione da dati preesistenti, il test genera dinamicamente tutte le entità necessarie, creando un nuovo progetto, una nuova issue e un tag univoco da utilizzare successivamente come criterio di ricerca.

Flusso di Esecuzione

Il test viene eseguito secondo la seguente sequenza operativa:

1. L'utente accede alla dashboard dei progetti dell'applicazione.
2. Per garantire l'isolamento del test, viene creato un nuovo progetto utilizzando un nome generato dinamicamente.
3. Prima dell'invio del form di creazione del progetto, viene registrato un listener per intercettare la richiesta HTTP `POST` diretta all'endpoint responsabile della creazione del progetto.
4. Il progetto viene creato e il test verifica il corretto completamento dell'operazione.
5. Una volta completata la creazione del progetto, la pagina viene ricaricata e sincronizzata con il backend.
6. Il test individua il progetto appena creato e accede alla relativa area di gestione delle issue.

7. Tramite il pulsante dedicato viene aperto il form di creazione di una nuova issue.
8. Vengono compilati tutti i campi obbligatori della issue, inclusi titolo, descrizione, priorità, tipologia e stato.
9. Viene aggiunto un tag univoco generato dinamicamente, che verrà successivamente utilizzato come criterio di ricerca.
10. Prima dell'invio del form viene registrato un listener per intercettare la richiesta HTTP POST associata alla creazione della issue.
11. Il form viene inviato e il test attende la risposta del server.
12. Dopo la creazione della issue, l'applicazione reindirizza l'utente alla pagina contenente l'elenco delle issue del progetto.
13. Il campo dedicato alla ricerca per tag viene compilato con il valore del tag precedentemente associato alla issue.
14. La ricerca viene avviata simulando la pressione del tasto **Enter**.
15. Il sistema esegue il filtraggio delle issue e aggiorna i risultati mostrati nell'interfaccia utente.
16. Il test verifica la presenza della issue creata all'interno dei risultati della ricerca.
17. Infine, vengono recuperati i titoli delle issue visualizzate e confrontati con il titolo della issue associata al tag ricercato.

Assertions

Il test è considerato superato esclusivamente se tutte le seguenti condizioni risultano verificate:

- La creazione del progetto preliminare viene completata con successo.
- Il server restituisce il codice di stato HTTP 201 **Created** in risposta alla richiesta di creazione del progetto.
- La creazione della issue necessaria al test viene completata correttamente.
- Il server restituisce il codice di stato HTTP 201 **Created** in risposta alla richiesta di creazione della issue.
- Il tag specificato viene correttamente associato alla issue durante la fase di creazione.
- La funzionalità di ricerca per tag accetta il valore inserito dall'utente ed esegue correttamente il filtraggio.
- La issue associata al tag ricercato risulta visibile nei risultati restituiti dall'applicazione.

- L'elenco delle issue visualizzate contiene effettivamente il titolo della issue creata durante il test.
- Il titolo recuperato dall'interfaccia coincide con quello associato al tag utilizzato come criterio di ricerca.

Nel complesso, il test realizza una verifica End-to-End della funzionalità di ricerca delle issue tramite tag, validando l'intero flusso applicativo che coinvolge la creazione delle entità necessarie, l'associazione dei tag alle issue, l'elaborazione della richiesta di ricerca da parte del backend e la corretta visualizzazione dei risultati filtrati nell'interfaccia utente.

Capitolo 14

Test di usabilità

L'efficacia del software non si misura esclusivamente dalla robustezza della sua architettura backend o dall'assenza di bug, ma anche dalla sua capacità di offrire un'esperienza di uso fluida e intuitiva per l'utente finale.

Per valutare questo aspetto fondamentale, è stata condotta una **Expert Review**, una metodologia di analisi della *UserExperience* affidata a valutatori esperti.

L'obiettivo di questa revisione è stato quello di esaminare l'interfaccia grafica e i flussi di navigazione dell'applicazione, identificando potenziali criticità.

L'analisi è stata strutturata attorno a le **10 Euristiche di Usabilità di Nielsen**.

Di seguito viene riportata la checklist di domande e criteri guida utilizzati dagli esperti durante la fase di auditing del sistema per ciascuno dei dieci pilastri.

1. Visibilità dello stato del sistema

- Il sistema informa sempre l'utente di cosa sta succedendo?
- Quando un'azione va a buon fine, l'utente riceve un feedback visivo immediato?

2. Corrispondenza tra sistema e mondo reale

- Il linguaggio, le icone e i testi utilizzati sono familiari all'utente?
- Le icone utilizzate hanno il significato che l'utente si aspetta nel mondo reale?

3. Controllo e libertà utente

- Se l'utente fa un errore ha a disposizione una "via d'uscita" chiara per annullare l'azione
- È facile uscire da una schermata di un modale o da un menu a comparsa senza essere bloccati?

4. Consistenza e standard

- Gli elementi grafici (colori, font) sono uguali in tutte le pagine del software?
- Il sistema segue le convenzioni standard del web/delle applicazioni (es. il logo in alto a sinistra riporta alla home, il profilo è in alto a destra)?

5. Prevenzione degli errori

- Il sistema fa il possibile per evitare che l'utente commetta un errore prima ancora che questo avvenga
-

6. Riconoscimento piuttosto che ricordo

- Le informazioni necessarie per completare un'operazione sono visibili sullo schermo, o l'utente deve fare lo sforzo di ricordare cosa ha fatto nella schermata precedente?

7. Flessibilità ed efficienza d'uso

- Le azioni frequenti richiedono pochi clic o passaggi?

8. Estetica e design minimalista

- Le schermate contengono solo le informazioni strettamente necessarie?
- La gerarchia visiva (dimensione dei titoli, contrasto dei colori) guida l'occhio dell'utente verso azioni principali?

9. Aiuto all'utente nel riconoscere errori

- I messaggi di errore sono scritti in un linguaggio comprensibile e umano o mostrano codici tecnici grezzi?

14.0.1 Esito della analisi tramite Expert Review

L'applicazione dei criteri della Expert Review ha permesso di evidenziare alcune aree di miglioramento nell'interfaccia frontend. Di seguito vengono apportati i principali interventi apportati al sistema.

- È stato integrato un sistema globale di notifiche asincrone tramite **Toastr Notification**. Ogni operazione di creazione e modifica genera un feedback visivo immediato.
- È stata inserita l'icona della matita in corrispondenza della sezione di modifica del progetto. Trattandosi di un pattern convenzionale, l'utente è in grado di associare istantaneamente l'elemento grafico alla rispettiva funzionalità senza sforzo cognitivo.
- All'interno di ogni finestra modale dell'applicazione è stato introdotto un pulsante **Annulla**. Questo garantisce all'utente una via d'uscita chiara per revocare l'azione.
- È stata introdotta una logica di controllo condizionale sul frontend: nel momento in cui una issue passa allo stato closed, la sezione relativa all'inserimento dei commenti viene **automaticamente disattivata** e inibita all'utente. In questo modo il sistema previene l'errore a monte, congelando lo stato del ticket ed evitando interazioni ridondanti o non coerenti con il ciclo di vita della issue.

Capitolo 15

Esperimenti con utenti reali

A integrazione delle valutazioni precedenti, il processo di validazione del sistema ha previsto una fase di **sperimentazione controllata con utenti reali**.

L'obiettivo di questa attività è stato quello di valutare l'effettiva usabilità, l'intuitività dei flussi e la curva di apprendimento dle software.

15.1 Candidati

Il gruppo di sperimentazione è stato composto attingendo a soggetti che operano quotidianamente all'interno di aziende del settore software. Questo ha permesso di confrontarsi con utenti aventi già familiarità con le logiche di sviluppo, sebbene appartenenti a realtà diverse: alcune di queste aziende, infatti, non adottano ancora internamente uno strumento strutturato per la gestione e il tracciamento delle issue, rendendo il test particolarmente indicativo per valutare l'impatto del software su flussi di lavoro non ancora digitalizzati.

Nel dettaglio, il campione è composto da tre candidati, con i seguenti profili:

15.1.1 Definizione dei task

L'architettura prevede che solo un account con privilegi di Amministratore (Admin) abbia l'autorità di registrare e configurare nuovi profili utente all'interno della piattaforma.

Pertanto, per l'esecuzione del test, si è provveduto a generare direttamente a sistema le utenze destinate ai partecipanti, fornendo a ciascuno di essi le relative credenziali di accesso preconfigurate. I particolare due da Amministratore e due utenze standard.

1. Compiti assegnati ai amministratore:

- **Task A1:** Aggiunta di un nuovo utente al sistema
- **Task A2:** Creazione di un nuovo progetto

2. Compiti assegnati ai profili Utente Standard:

- **Task U1:** Apertura e creazione di una nuova issue
- **Task U2:** Interazione e Collaborazione tramite Commenti

Candidato	eta'	Ruolo
Candidato 1	23	Sviluppatore software Junior abituato a lavorare in team.
Candidato 2	24	Frontend Designer focalizzato sull'interfaccia e l'interazione.
Candidato 3	29	Project Manager, figura focalizzata sulla supervisione dei processi, l'assegnazione dei task e il monitoraggio dello stato di avanzamento dei progetti.
Candidato 4	43	ProjectManager che non ha mai utilizzato un'app di issue-trac

Tabella 15.1: Tabella dei candidati per testare l'usabilità

15.1.2 Risultati dell'esperimento

Candidato	Caso d'uso	Tempo impiegato	N°errori
Candidato 1	Creazione Nuova Issue	1m14s	0
	Interazione tramite Commenti	33s	0
Candidato 2	Creazione Nuova Issue	2m	1. cliccando invio nei tag ha postato la issue senza volerlo.E ha dovuto ricominciare.
	Interazione tramite Commenti	1m	0
Candidato 3	Aggiunta di un nuovo utente al sistema	49s	0
	Creazione di un nuovo progetto	36s	0
Candidato 4	Aggiunta di un nuovo utente al sistema	1m14s	0
	Creazione di un nuovo progetto	34s	0

Tabella 15.2: Metriche raccolte durante i test di usabilità

15.2 Sondaggio finale

Al termine dell'esperimento, ai partecipanti è stato sottoposto il seguente sondaggio:

Sondaggio BugBoard
Valutazione del software BugBoard

* Indica una domanda obbligatoria

Quanto hai trovato complesso il software? *

1 2 3 4 5

Quanto ritieni sia facile, per un nuovo utente, imparare a usare l'applicazione? *

☐ Per nulla facile
☐ Poco Facile
☐ Facile
☐ Molto Facile

Hai riscontrato bug nel sistema durante l'utilizzo? *

☐ Nessuno
☐ Pochi
☐ Molti

Riutilizzeresti ancora l'app? *

☐ No
☐ Non Penso
☐ Penso di sì
☐ Sì

Quanto è stata di tuo gradimento l'app? *

☐ Per niente
☐ Non molto
☐ Abbastanza
☐ Molto

Che miglioramenti faresti?

La tua risposta

Figura 15.1: Sondaggio test usabilità

Dai risultati di quest'ultimo, le migliori proposte dagli utenti sono le seguenti

- **Migliorare le notifiche:** Renderle più chiare e brevi
- **Migliorare il pulsante di aggiunta foto nel form issue:** Renderlo più visibile.

15.3 Conclusioni

Dalle risposte raccolte emerge che gli utenti hanno valutato positivamente le funzionalità principali dell'applicazione. Le osservazioni più frequenti riguardano la gestione delle notifiche e dei messaggi di errore, che dovrebbero essere resi più evidenti e brevi per facilitare l'individuazione e la correzione di eventuali problemi.

Infine, è stata segnalata una difficoltà nell'individuazione della funzionalità di caricamento immagini durante la creazione di una nuova issue. Sebbene la funzione sia già disponibile, non risulta sufficientemente evidente all'utente, che potrebbe non comprenderne immediatamente la presenza o la modalità d'uso.

Nel complesso, il feedback raccolto ha permesso di individuare possibili evoluzioni, fornendo indicazioni utili per le future attività di manutenzione e miglioramento del sistema.